



A Survey of Methods for Automated Quality Control Based on Images

Jan Diers¹ · Christian Pigorsch¹

Received: 18 October 2022 / Accepted: 17 May 2023 / Published online: 10 June 2023
© The Author(s) 2023

Abstract

The role of quality control based on images is important in industrial production. Nevertheless, this problem has not been addressed in computer vision for a long time. In recent years, this has changed: driven by publicly available datasets, a variety of methods have been proposed for detecting anomalies and defects in workpieces. In this survey, we present more than 40 methods that promise the best results for this task. In a comprehensive benchmark, we show that more datasets and metrics are needed to move the field forward. Further, we highlight strengths and weaknesses, discuss research gaps and future research areas.

Keywords Quality control · Anomaly detection · Image analysis · MVTec anomaly dataset

1 Introduction

The possibility to detect anomalies in workpieces or materials is a central task for quality assurance. In many cases, a defect can be detected based on an image. The analysis of images using computers, the field of computer vision, has achieved significant success in recent years. This is due in large parts to the existence of large datasets such as ImageNet (Deng et al., 2009) or Open Images (Krasin et al., 2017), which contain several million images. Each image is associated with a label, which allows training neural networks that learn a mapping from images to labels.

Quality control aims to detect defects in products. Defects have the inherent property that they are rare. Collecting a dataset of thousands of images, as can be done for other computer vision tasks, is rarely possible in this situation. The few datasets that have been published for quality control (Carrera et al., 2017; Wieler & Hahn, 2007) contain only a few low-resolution images. As a result, there has been little attention from the computer vision community to the field of quality control.

Recently, this has changed. Driven by the publication of the MVTec Anomaly Dataset (MVTec AD) (Bergmann et al.,

2021a), a variety of methods has been proposed that promise successful detection of defects. The methods operate without labels to detect defects in products. This offers two advantages: First, there is no need for manual labeling to apply the methods. Human labeling, where skilled workers annotate each image regarding the anomaly, is thus not required. Second, no assumptions need to be made about what types of anomalies will occur *in the future*. This has the pleasant effect that the methods can also detect currently unknown defects.

This work aims to present a comprehensive overview of modern methods proposed for the task of quality control. For this purpose, we present more than 40 methods, provide a benchmark based on their results and group them according to their approach. All methods have in common that they do not require labels to detect the defects in products. For the application, it is sufficient to have a collection of images that do not contain a defect. The methods learn a model of the normal product to identify defects.

We group the works according to their approach. A large group of methods works with pre-trained neural networks as feature extractors and feature distances. These methods are presented first. Then we present methods based on deep one-class classification. Other groups of methods we present are based on student-teacher learning, artificially generated anomalies, dimension reduction or GANs. In the last section, we present methods based on Normalizing Flows, an emerging class of anomaly detection models.

✉ Jan Diers
jan.diers@uni-jena.de

Christian Pigorsch
christian.pigorsch@uni-jena.de

¹ Friedrich-Schiller-University Jena, Fürstengraben 1, 07743
Jena, Germany

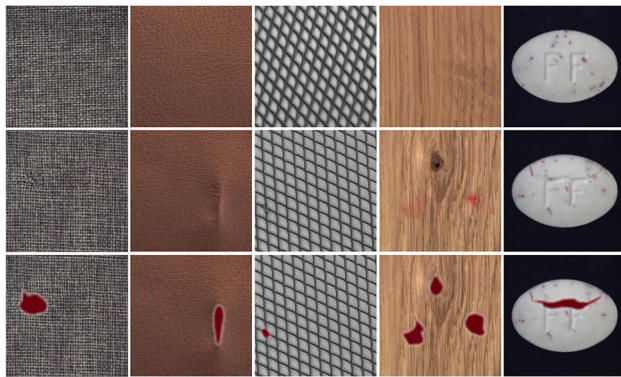


Fig. 1 Can you spot the defect? The methods presented in this survey help to detect defective products by using images. The methods work unsupervised, meaning no prior knowledge of the anomaly is assumed. An example of the output of the methods is shown in the bottom row. Top row: normal product without defect. Middle row: product with defect. Bottom row: example of the output of a method that detects the defect shown in the middle row. The images are taken from the MVTec anomaly dataset (Bergmann et al., 2021a)

The methods are ranked according to their performance in a benchmark. Performance is compared for both segmentation and detection. Segmentation refers to a pixel-by-pixel annotation of the image to see if there is an anomaly at that pixel. Detection detects anomaly at the image level, checks more generally whether there is an anomaly on the image as a whole. Our work concludes with a discussion of the results, showing strengths and weaknesses of the approaches, as well as future research directions.

An alternative to segmentation models is to use methods of Explainable AI (XAI). XAI is a field of research that visualizes the results of machine learning in a way that is understandable to humans (Adadi & Berrada, 2018). A common approach is to remove parts of the image and observe how the model's output changes. While this does not explain the model, it can visualize which input factors lead to the model's decision. This is a sensitivity analysis with respect to the input factors (Xu et al., 2019). In principle, methods for XAI are also suitable for visualization in defect detection. They can be used to visualize models that do not perform image segmentation. We will take up this aspect in the discussion section.

2 Relationship to Other Research in Outlier Detection

2.1 Distinction from Other Outlier Detection Problems

The methods proposed for the MVTec anomaly dataset join the wide range of outlier detection methods. The definition of an outlier is fuzzy, which makes the research field

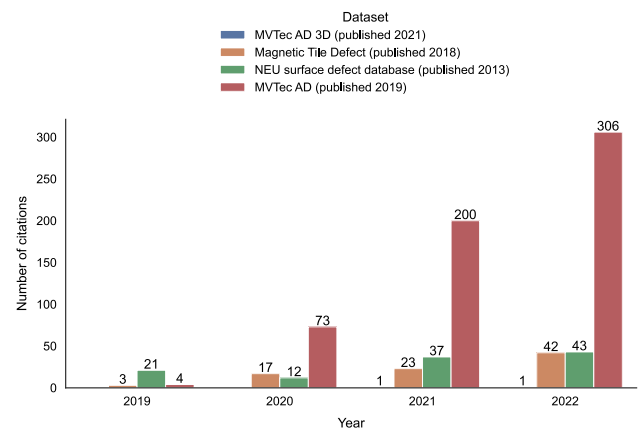


Fig. 2 The number of citations for the respective datasets over time. For MVTec AD, it is visible that this dataset clearly influences research in the field. This also justifies the objective of this survey, which considers methods on MVTec. The citation counts were obtained using the exact keywords in Google Scholar

extremely broad. A key feature that distinguishes industrial defect detection from other areas of outlier detection is the fact that industrial defect detection requires the localisation of the anomaly. Traditional anomaly detection algorithms operate at the image level. Therefore, they are rarely able to segment the image with respect to the anomaly. In the following, we make a detailed distinction between the problem at hand and related applications of outlier detection. For a high-level overview, see Table 1 and Fig. 3.

(I) Time series, tables, and other structured data

Conventional and widely used outlier detection methods were developed for structured data. Images such as those considered in this survey cannot be processed directly. Conventional outlier detection methods include one-class support vector machines (Schölkopf et al., 1999), local outlier factor (Breunig et al., 2000), methods based on neighborhood

Table 1 Overview of different settings in the field of anomaly detection

	Input is image	Nominal/outlier have similar semantics	Outliers are taken from real world
(I)	×	○	○
(II)	✓	○	✓
(III)	✓	✓	×
(IV)	○	✓	✓
(V)	✓	✓	✓

Traditional methods expect structured input, while in the field of images, data sets with natural anomalies are rare. Defect detection datasets are characterized by the fact that anomalies are not synthetically generated and do not significantly change the semantics of the image. In contrast to medicine, in which multiple input channels are common, the methods presented here only process RGB input. ✓ indicates true, ○ indicates partially true, × indicates not true. (I)–(V) refers to the numbering of the sections in this chapter

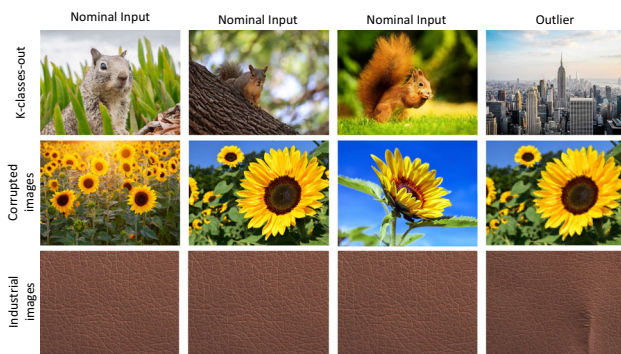


Fig. 3 Shown are different settings for anomaly detection on images. *K*-classes-out treats semantically different concepts as anomalies. Corrupted images represent synthetic anomalies, such as Gaussian noise or blur. In the case of industrial defects, the anomalies are of natural origin and can only be detected at a local level

graphs, PCA, and others (Hautamaki et al., 2004; Xu et al., 2010; Tax & Duin, 2004). However, methods have been proposed, for example, based on pre-trained neural networks as feature extractors, which make it possible to use the conventional methods with images.

(II) *K*-classes-out setting

Traditional benchmarks for outlier detection, which target structured data, are therefore not applicable to image processing. For this reason, it was established as a benchmark to take datasets from supervised learning and select a number of classes there that represent the outliers (Bergman & Hoshen, 2020; Ruff et al., 2018, 2020). This corresponds to the setting *k*-classes-out. For example, in the case of the MNIST dataset, it is possible to consider all digits from 0–8 as nominal instances, while images with the digit 9 must be recognized as outliers. While this is an example for outlier detection on images, the task is very different from subtle defects in industry. Using Cifar-10 as an example, if all images of cars are selected as nominal instances, then airplanes represent semantically different images. The whole image then shows a semantically different object in the form of the airplane. In the case of manufacturing defects, the semantics have not changed: An image of a production part with a defect is still semantically an image of a production part.

(III) *Corrupted images*

In the *k*-classes-out setting, the outliers are semantically different from the nominal points. However, it is also possible to create outliers for images that are semantically largely unchanged. Hendrycks and Dietterich (2019) have published datasets (ImageNet-C, ImageNet-P, CIFAR-10-P, CIFAR-100-P) for this purpose, which contain images from ImageNet, but change the quality of the image by corrupting and distorting the input. Mu and Gilmer (2019) publish the benchmark for MNIST. The semantics of these datasets remain unchanged, which is the core difference to the *k*-

classes-out setting. However, these are *synthetic* anomalies, which is different from the datasets in this paper.

(IV) *Medical images*

Medical images share many characteristics with defects in production parts. One difference is that in medicine, there is often a priori knowledge about the nature of the anomaly. This makes it easier to use supervised learning. However, unsupervised methods have also been proposed for medical images and can be considered for defect detection. Bergmann et al. (2021a) have included AnoGan, a medical method, in the benchmark. However, they note that AnoGan has great difficulty with objects that occur in different shapes or orientations in the dataset. This suggests that the methods may be suitable, but that adjustments are needed to better reflect the higher variance of product images. This will be addressed in the discussion of the methods.

(V) *Industrial defect images*

Industrial quality control images are characterized by the fact that the anomalies on the image can be very small and do not change the semantics of the image. Unlike corrupted images, these are real-world anomalies that have not been artificially created. The anomaly is usually only local, while other areas of the image do not allow the anomaly to be detected. Furthermore, the orientation of medical images tends to be more consistent than in quality control.

2.2 Relationship to Other Survey Articles

Due to the wide range of research in outlier detection, there are numerous survey articles. These include works that categorize different algorithms in their methodology Hodge and Austin (2004) or group them by type of application Singh and Shuchita (2012); Shahid et al. (2012). There is also work that is specifically tailored to different types of data. These include surveys on outlier detection based on time series data Gupta et al. (2014); Bl'azquez-Garc'ia et al. (2020), categorical data Taha and Hadi (2020), streaming data Thakkar et al. (2016) or medicine Chromiński and Tkacz (2010); Juliano Gaspar et al. (2011).

While these articles provide comprehensive overviews of methods and applications, they lack the algorithms of recent years, which are often based on deep neural networks. There are relatively recent deep learning review articles that classify methods in general Chalapathy and Chawla (2019); Pang et al. (2022); de Albuquerque Filho et al. (2022) or by specific algorithms Di Mattia et al. (2019), but these articles do not relate traditional methods to deep learning research. Ruff et al. (2021) fills this gap. To this end, they put traditional outlier detection methods in context with deep methods, and also benchmark different classical and deep methods on different datasets.

Ruff et al. (2021) also have a benchmark on the MVTec anomaly dataset. However, they do not consider any of the

methods proposed specifically for this problem. The benchmark also shows that generalist methods such as kernel density estimation do not perform as well as the methods in this survey. Zheng et al. (2022) also benchmark on the MVTec dataset, but the benchmark is not comprehensive with 13 methods. In contrast to other work, we present methods that (i) visualize and thus explain anomalies on the image and (ii) provide reliable results for subtle anomalies found in industrial defect detection.

3 Background, Explainability of the Results, Datasets and Metrics

3.1 Background

The digitization of production is part of daily life in industry. Smart factories, e-manufacturing and Industry 4.0 aim to digitize processes and production in factories. One of the greatest potentials of digitization is machine learning: Work tasks that previously required human skills can also be performed by computers and robots thanks to modern methods (Lilhore et al., 2022; Gourisaria et al., 2021).

Machine learning and neural networks have set new standards in the field of computer vision in terms of the quality of the analyses. In this way, they have become the standard method currently used on this type of data. This is also true for quality control: visual inspection of the quality of a workpiece can be automated using these methods.

Automation of quality control has a high priority, since for any production this step is of central importance. Equally, as machine capacity increases and depending on the skill of the workforce, it is more difficult to ensure high-quality inspections (Kang et al., 2018). Human–machine collaboration offers opportunities for this: computer vision algorithms can help human workers to detect defects and anomalies in manufactured parts, thus providing high reliability of quality control.

In a digitized factory, human labor remains important. Hozdić (2015) states that many production steps already replace human labor, but human labor remains an important factor for production. Zuehlke (2010) supports the human factor in production and states that digital systems of production should support human labor. This illustrates that algorithms that support humans in making a decision represent progress for factories. To achieve this, however, humans and machines must be able to communicate with each other in a simple way, so the machine should not represent a pure black-box decision.

It is a possibility to open the black-box by visualizing the algorithm's decision on the image. This is the objective of the explainability of the results, which will be taken up in the following section.

3.2 Explainability of the Results

Anomalies in products can be very subtle and difficult to detect on superficial inspection (see Fig. 1). Methods differ according to whether they can localize the anomaly. If the method can highlight the anomaly in the image, this is an advantage for practical use: reliable verification of the defect by a skilled worker is much easier.

The decision can be visualized in different ways on the image. These are shown in Fig. 4. The score and the classification of the entire image do not allow any conclusions regarding the position on the image where an anomaly or a defect could be found. In contrast, for the bounding box and segmentation, it can be rapidly determined whether the algorithm is correct in its decision. In this work, we consider methods that segment images. The segmentation indicates for each pixel of the image whether the pixel represents a defect. If there are enough pixels that are marked as defects, the image as a whole is classified as a defect. In this way, the methods can both make a statement for the entire image whether a defect is visible, and also justify where the defect can be seen.

Image segmentation is an established discipline in machine learning. The vast majority of methods work based on annotated datasets, where each image and pixel has been annotated to map the class at pixel level. This makes it possible to learn models that perform image segmentation. An overview is provided by Minaee et al. (2021).

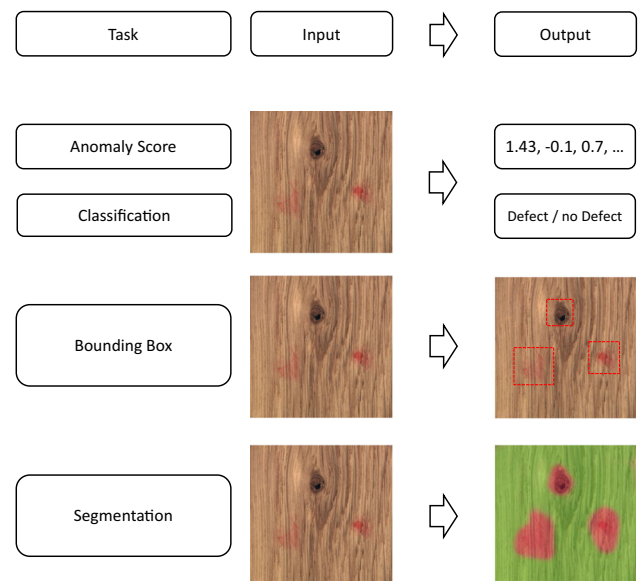


Fig. 4 Differences between anomaly score, classification, bounding box prediction and anomaly segmentation. While the anomaly score and the classification do not provide any information about the location of the anomaly, the bounding box and the segmentation can be used to quickly determine whether the algorithm is correct

For defect detection, existing methods have limited applicability. This is because defects may be unknown a priori if a previously unknown defect arises in a workpiece. The detection of new anomalies is then unreliable. Unsupervised methods that do not require labeled data exist (Kim et al., 2020; Xia & Kulis, 2017; Croitoru et al., 2019), but have only recently begun to receive more attention from the community (Minaee et al., 2021).

3.3 Datasets

In the following part, we present datasets that contain images of industrial manufactured parts. These are MVTec AD, magnetic tile surface defects dataset, NEU surface defect database, MVTec 3D and OLP. In terms of citations over the years, MVTec AD is the most prominent dataset, which has significantly influenced research in the field. See Fig. 2. A distinguishing feature between the anomaly detection datasets presented here and other datasets is that the datasets presented here have a pixel precise annotation where in the image a defect is present. In other anomaly detection datasets, this annotation is missing or not possible due to the nature of the task.

MVTec AD (Bergmann et al., 2021a) contains more than 5000 images of 15 different product categories.¹ The product categories include textures and objects. About 4000 images are without defects, the remaining images contain at least one defective region. There are 73 different types of defects included.

Each of the images provides a pixel-precise annotation at which location a defect is located. The images of the training dataset generally do not contain a defect. The objective is to develop methods that detect defects without being trained on defect images. The resolution of the images is mostly 1024×1024 , with occasional products shown at lower resolution. Sample images of the products can be found in Fig. 1. A detailed overview of the dataset is provided in Table 2.

An alternative to MVTec AD is the dataset published by Huang et al. (2020).² It is about the detection of defects in surfaces of magnetic tiles. Defects in magnetic tiles are difficult to detect because each tile has different texture, the magnetic tiles have different shapes, and are inconsistently illuminated. The dataset addresses these challenges and includes 5 different types of anomalies. These are: Holes, fractures, cracks, fraying, and unevenness in the magnetic tiles. The dataset contains more than 1300 images, about 950 of which have no defect. The remaining 350 images each contain a defect.

One challenge is that images are included at different resolutions and the images have different heights or widths.

A similar dataset to the magnetic tiles represents the NEU surface defect database (Song & Yan, 2013).³ It contains six types of typical surface defects in hot-rolled steel strip. The dataset includes a total of 1800 gray images, each of size 200×200 pixels. Exactly 300 images of each defect class are included, and multiple defects may be visible on an image at the same time. Both annotated boxes and ground truth are included as labels.

A more recent dataset represents MVTec 3D (Bergmann et al., 2021b).⁴ Following the widely used MVTec AD in 2D, the 3D dataset contains anomalies of various workpieces in three-dimensional representation. Among them are images of carrots, cookies, dowels, cable gland, bagels, and others. In total, the dataset includes more than 4000 scanned products, each with precise annotation regarding the defect.

Rippel et al. (2022) propose a dataset named OLP, which contains 38 patterned fabrics.⁵ Each piece of fabric is shown twice: Once with exposure from the front and once with exposure from the back. These image-pairs represent the standard in today's manufacturing to perform quality control. Like the other datasets, OLP is annotated at image level and pixel level, bounding boxes of defects are also included.

Other datasets used include BeanTech AD (BTAD)⁶ (Mishra et al., 2021) and Kolektor Surface-Defect Dataset (KSDD)⁷ (Tabernik et al., 2020). BTAD contains 1799 images of three different products at resolutions 600×600 to 1600×1600 . KSDD contains 399 images with microscopic fractions or cracks. These were observed on the surface of the plastic embedding in electrical commutators. Both BTAD and KSDD have pixel-precise annotations, KSDD also provides a variant with bounding boxes for the defect.

3.4 Metrics

The metrics for the task differ depending on whether it is evaluated at the image level or the pixel level. For the image level, the ROC-AUC value is the most widely used and is reported in all papers. However, other classification metrics can also be considered, although they are currently not widely used in the community (precision, recall, F1-score,...).

At the pixel level, it is also standard to report the ROC-AUC score. In addition, image segmentation metrics can be applied. An example is the Intersection over Union (IoU).

¹ Link to download: <https://www.mvtec.com/company/research/datasets/mvtec-ad>.

² Link to download: <https://github.com/abin24/Magnetic-tile-defect-datasets>.

³ Link to download: <http://faculty.neu.edu.cn/songkc/en/zhym/263264/list/index.htm>.

⁴ Link to download: <https://www.mvtec.com/company/research/datasets/mvtec-3d-ad>.

⁵ Link to download: <https://github.com/ORippler/OLP-dataset>.

⁶ Link to download: <https://github.com/pankajmishra000/VT-ADL>.

⁷ Link to download: <https://www.vicos.si/resources/kolektorsdd/>.

Table 2 Overview of the MVTec anomaly data set

Category	Train images	Test images without defect	Test images with defect	Different defect types	Image size
<i>Textures</i>					
Carpet	280	28	89	5	1024
Grid	264	21	57	5	1024
Leather	245	32	92	5	1024
Tile	230	33	84	5	840
Wood	247	19	60	5	1024
<i>Objects</i>					
Bottle	209	20	63	3	900
Cable	224	58	92	8	1024
Capsule	219	23	109	5	1000
Hazelnut	391	40	70	4	1024
Metal nut	220	22	93	4	700
Pill	267	26	141	7	800
Screw	320	41	119	5	1024
Toothbrush	60	12	30	1	1024
Transistor	213	60	40	4	1024
Zipper	240	32	119	7	1024
Total	3629	467	1258	73	–

The dataset includes more than 5000 images in high resolution. The training data is without defects, while in the test data both defect and non-defect images appear. In total, across all products, defects can be seen on 1258 images

For all pixels P on which an anomaly was predicted, and all pixels G marked as an anomaly in the ground truth, the IoU for an image is calculated as:

$$IoU(P, G) = \frac{P \cap G}{P \cup G}$$

If only pixel values are considered, small defects lose their significance in the metric. A model that only detects large defects and no small defects can achieve good results in this way. For this reason, other metrics should be considered that recognize contiguous pixels as an anomaly. Bergmann et al. (2021a) propose the PRO metric for this purpose. P still denotes all pixels that were predicted as anomalous by the model. C_k denotes pixels that were marked as anomalous in the ground truth and represent a connected component k . The PRO metric is then calculated as:

$$PRO = \frac{1}{N} \sum_i \sum_k \frac{P \cap C_k}{C_k}$$

A major difference between ROC-AUC and the PRO score is that ROC-AUC does not require a threshold, whereas to calculate the PRO, a threshold must be defined to convert the anomaly score into a binary prediction. For this reason, it is common to vary the threshold and calculate several PRO scores for each threshold. The result can be plotted and the area under the curve calculated, resulting in the AUC-PRO metric. Bergmann et al. (2021b) point out that the area should

only be considered up to a false positive rate of 0.3, which has also become the standard in the literature. This is because the anomalous regions are comparatively small in relation to the number of images, which would otherwise lead to an imbalance in the metric.

4 Methods for Detecting Defects in Quality Control

4.1 Notation and Terminology

We note an image x as $x \in \mathbb{R}^{H \times W \times C}$, where H , W , and C are the height, width, and number of channels of the image, respectively. The training dataset is represented by X , where X consists of all samples that are used during training: $X = \{x_i \mid x_i \in \text{trainset}\}$. It is important to note that the presented methods are trained on data that does not contain anomalies.

An image x is processed by a neural network, which we denote as a function f . $f_1, f_2, f_3, \dots, f_n$ represents f at different stages to extract features of different scales. f_n refers to the last stage, i.e., the entire network without the fully connected part. The output of $f_i(x)$ is a vector called embedding: $e = f_i(x)$. Many methods compute the distances between the embedding of x and the embeddings from the training dataset. We abbreviate the k -nearest neighbors of x as $N_k(x)$, so that $N_k(f_i(x))$ defines the nearest neighbors in the embedding

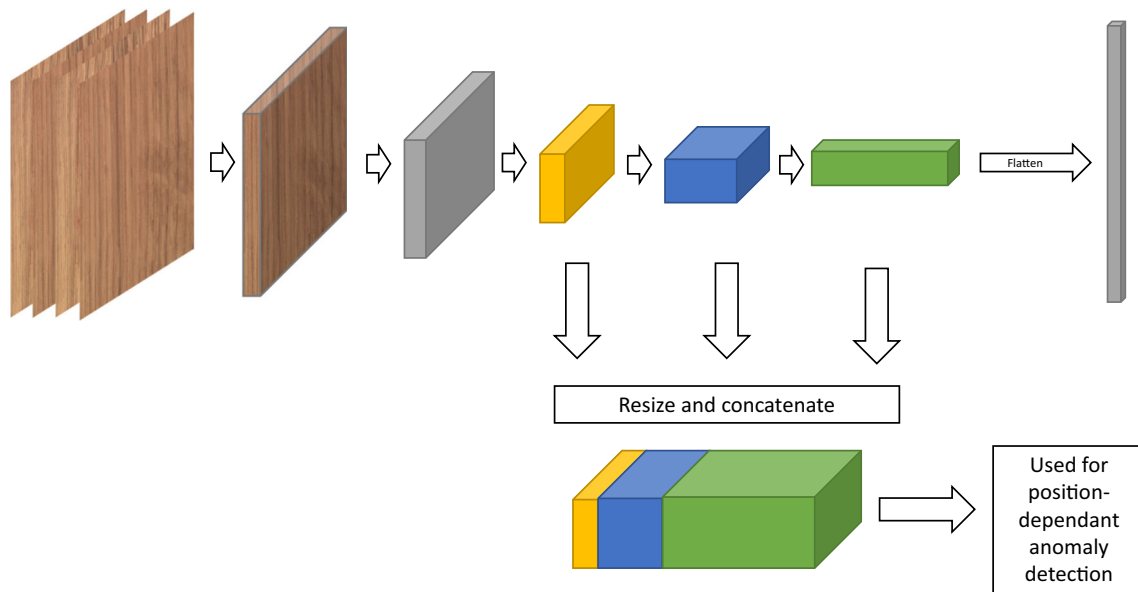


Fig. 5 Schema for methods based on feature distances. Pre-trained neural networks are used to extract features at different stages. The processing of features depends on the method, common methods are based on Mahalanobis or L2 distance

space for stage i . Unless otherwise stated, distances always refer to the distance of a sample to the training dataset.

4.2 Methods Based on Feature Distances

These methods use pre-trained neural networks as feature extractors. The extracted embeddings of an image are then compared with other embeddings from the past to detect the defect in the image patch.

The extracted features of pre-trained neural networks provide an effective way to detect anomalies in image data (Bergman et al., 2020; Napoletano et al., 2018). The neural network is used to map the image to a vector. This vector contains semantically rich information of what can be seen in the image. Afterward, conventional anomaly detection methods (for example, distance to k-nearest neighbors or one-class support vector machines) are used to evaluate the vector in terms of normality.

The vector obtained using the neural network is called embedding. Embeddings can be extracted for the entire image or for parts of the images that are called patches. The layer of the neural network that is used to produce the embedding depends on the method. For anomaly detection on image level, often the output of only one layer is considered, so that the last convolutional layer is used and the fully connected part of the neural network is ignored.

For pixel-level anomaly detection, many works (Cohen & Hoshen, 2020; Roth et al., 2021; Defard et al., 2021) argue that only one layer is too coarse for accurate anomaly segmentation. The later layers (close to the classification layer) lose spatial connectivity because the height and width of the

feature maps are much smaller than the size of the input image. In turn, the later layers contain semantically mature concepts that abstract more from the details of the image. The early layers of the neural network (close to the input layer) contain many details of the image and are better at spatially associating objects in the image. The feature maps are larger than later in the network. Consequently, these works combine the output from multiple layers to get the benefits from all layers. This is shown schematically in Fig. 5. The relationship of the methods is shown in Fig. 6.

An example of anomaly detection on image level is represented by the work of Rippel et al. (2020). Features $f_1(x), \dots, f_9(x)$ based on an EfficientNet (Tan & Le, 2019) are used and a multivariate Gaussian distribution is estimated on the embeddings. The Mahalanobis distance is used as the distance measure and anomaly score for an image:

$$M_i(x) = \sqrt{(f_i(x) - \mu_i)^\top \Sigma_i^{-1} (f_i(x) - \mu_i)} \quad (1)$$

where $i = 1, \dots, 9$ refers to the level f_i . The covariance is estimated based on the features extracted on $f_i(X)$, using shrinkage for a stable and nonsingular estimation of the matrix. Rippel et al. (2020) also show how a threshold can be selected under the assumption of the Gaussian distribution so that a desired false positive rate is not surpassed.

The work of Napoletano et al. (2018) uses embeddings from different parts of the image to achieve a segmentation of the image. For this, the image is cut into patches to generate an embedding for each patch. Based on the training dataset, which does not contain anomalies, embeddings are generated that represent the normal distribution. These are stored in a

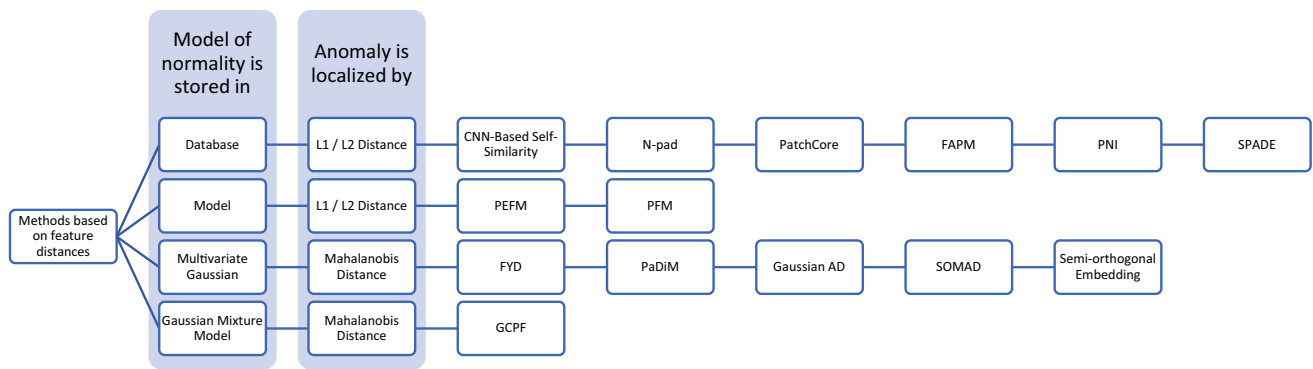


Fig. 6 Overview of the functionality and relationship of feature distance-based methods. Since the models are trained only on images without defects, they use different strategies to store the distribution of nominal images. At inference time, the normal image is compared to the current image to detect anomalies

database. For anomaly detection in new images, the embeddings for each patch are matched with the embeddings from the database. This is done by calculating the distance between the embeddings. If the distance of a patch to the patches from the database is too large, then the patch is declared as an anomaly.

This approach has the disadvantage that each patch has to be processed individually by the neural network to obtain the embedding. This is time-consuming. Likewise, each patch must be resized to have the correct input size for the neural network. Since the patches are relatively small to allow accurate segmentation, resizing small patches to larger images comes with a loss of image quality.

SPADE (Cohen & Hoshen, 2020) uses both the last convolutional layer and intermediate layers to detect anomalies. The method proceeds in two steps. First, the embedding of the entire image is taken and checked if it is an anomaly. This is done by calculating the Euclidean distance between the embedding of the image $f_n(x)$ and the k -nearest embeddings of the training dataset:

$$d(x) = \frac{1}{k} \sum_{e \in N_k} \|e - f_n(x)\|^2 \quad (2)$$

If the distance $d(x)$ is too large, then the image is identified as an anomaly and the second step is started.

The second step of SPADE aims to locate the anomaly in the image. For this purpose, the embeddings of three intermediate feature maps $f_1(x)$, $f_2(x)$, $f_3(x)$ are taken. These represent the anomalies at pixel level. The embeddings are resized to one size and stacked. The embeddings at the pixel level are then compared with the embeddings of the training data in the same way as in the first step and the distances are calculated. A pixel p in an image x is denoted as $(x)_p$ and has k -nearest neighbors $N_{k,p}$. The anomaly score for p is then calculated as:

$$d_p(x) = \sum_{e \in N_{k,p}} \|e - (f_{1...3}(x))_p\|^2 \quad (3)$$

whereby $(f_{1...3}(x))_p$ stands for the resizing and concatenation of the separate feature maps f_1 , f_2 and f_3 and take the value for pixel p . If the distance is too large, then p is marked as an anomaly. Afterward, a Gaussian Filter is applied to the resulting image to smooth the results.

PatchCore (Roth et al., 2021) builds on the work of SPADE. One drawback of SPADE is the two-step detection. First, an image must be detected as an anomaly on image level, and then the anomaly must be localized in the image. One of the reasons for this is that the search of k -nearest neighbors for each image is comparatively slow. PatchCore, analogous to SPADE, also works with a memory bank obtained on the training data. However, the memory bank is reduced in size by combining similar points into one. The reduced database size is called the core set. Since the number of query points is less, the nearest neighbor search is faster. For this reason, PatchCore does not rely on SPADE's two-step approach.

If C_{ore}^{set} represents the function that reduces the nearest neighbors to a core set, then the anomaly score for a patch p is calculated as follows

$$d_p(x) = \sum_{e \in C_{ore}^{set}(N_{k,p})} \|e - (f_{1...3}(x))_p\|^2 \quad (4)$$

which also allows segmentation of the image. The anomaly score on image level is the maximum anomaly score of a single patch.

Another difference between PatchCore and SPADE is the consideration of neighboring pixels. PatchCore uses an AveragePooling to bring the different feature maps to the same size. This averages the embeddings from neighboring pixels, which is similar to local smoothing as SPADE does. This allows PatchCore to perceive more local context than it is the case with SPADE.

PaDiM (Defard et al., 2021) joins the family of PatchCore and SPADE. The method follows a similar strategy, but uses

a parametric model instead of neighborhood-based anomaly detection. The vectors for each pixel are obtained based on ResNets (He et al., 2015) and EfficientNets (Tan & Le, 2019), respectively. The feature maps are resized to be stacked. PaDiM uses a multivariate Gaussian distribution in place of SPADE's kNN method to model the normal distribution of embeddings. Similar to PatchCore, PaDiM also considers local neighborhoods, since the covariance matrix of the Gaussian distribution maps dependencies between embeddings.

Let the resized and concatenated embeddings of stages 1, 2 and 3 be $e_p = (f_{1...3}(x))_p$. The anomaly score for a pixel p is then defined as

$$M_p(x) = \sqrt{((e_p - \mu_p)^\top \Sigma_p^{-1} (e_p - \mu_p))} \quad (5)$$

The difference to (1) is that (1) considers the distance for a feature map, while (5) considers the distance per pixel. Also, in (5), multiple feature maps are considered at the same time.

The use of the multivariate Gaussian distribution has the advantage of being a parameterized model. The distance-based detection of SPADE and PatchCore requires that the memory bank is scanned for each anomalous image. This may take time. In contrast, PaDiM relies on the estimated Gaussian distribution and thus no longer needs to rely on past training data. Anomaly detection is done by estimating the Mahalanobis distance. This requires the mean and standard deviation of the Gaussian distribution. In turn, it is not necessary to scan all samples of the database, which significantly reduces the time required for inference.

Kim et al. (2022) also emphasize the slow inference time that can be encountered for feature embedding methods. They therefore propose an algorithm called Fast Adaptive Patch Memory (FAPM). Their model keeps a separate database for each patch and layer of the trained network. In this way, they can reduce the inference time by a factor of five, which results in a method that can process over 40 frames per second.

The work by Kim et al. (2021) is also based on the Mahalanobis distance. In this paper, the authors argue that it can be costly to compute the inverse covariance matrix that is required to compute the distance. The paper proposes a method to make a robust estimate of the inverse covariance matrix that reduces the computational complexity.

Traditionally, clustering algorithms have also been widely used to identify anomalies in datasets. Since the algorithms usually require structured data, Wan et al. (2022a) use embeddings based on a pre-trained network, similar to the approach of SPADE. A Gaussian Mixture Model is then estimated on the embeddings to cluster the available features. Inference of the anomaly is conducted using the Mahalanobis distance and the parameters of the Gaussian Mixture Model.

Li et al. (2021) propose to use self-organizing maps (SOM) with a memory block. For this, they extract features with a trained network to learn a SOM on the extracted features. The advantage here is that the SOM preserves the spatial mapping of pixels by design, which is not the case with some other methods based on pre-trained models. The anomaly score is also based on the Mahalanobis distance.

PFM (Wan et al., 2022b) constitutes a mixture of feature distance learning and reconstruction. PFM uses a trained model to generate embeddings at different scales. A second model is learned to reconstruct the mapping of the first model. The objective function is to minimize the Euclidean distance between the embeddings of the two models. The anomaly score is given by the distance between the embeddings. In Wan et al. (2022c), the authors extend their approach with positional encodings to provide better localization of the anomaly.

A challenge of the previously mentioned methods is that feature extraction is not translation invariant. That is, if the images are, for example, rotated, then the extracted features will also change and so will the neighborhood search and Mahalanobis distance estimation. Focus Your Distribution (Zheng et al., 2021) addresses this weakness: In a preprocessing stage, images are aligned in a self-supervised manner, and then features are extracted from them. In this way, images are created that have an approximate, normalized representation. The extracted features are further enhanced in the proposed method by using non-contrastive learning to extract improved semantic vectors for each image patch. Based on the improved vectors, Mahalanobis distance is used to locate anomalies.

Bae et al. (2022) also argue that the extracted features in the database do not contain information from which location of the image the feature was taken. However, for determining whether an area of the image is an anomaly, the location of the feature and its neighborhood is highly relevant. For this reason, they condition the embeddings on neighboring pixels, learning the neighborhood relationship with a multi-layer perceptron. In a second step, they use a refinement network, where synthetic anomalies are generated and inserted on the image to localize the anomalies even more precisely.

N-pad (Jang et al., 2022) also exploits information from neighboring pixels to determine the distribution of the inner pixel. During training, this involves determining a centroid set for each pixel, which is the nominal distribution for the pixel's neighborhood. At test time, the distance of the present image from the centroid set is determined to locate anomalies on the image. In a subsequent step, the authors improve their localization accuracy by running multiple, shifted variants of the image through their algorithm.

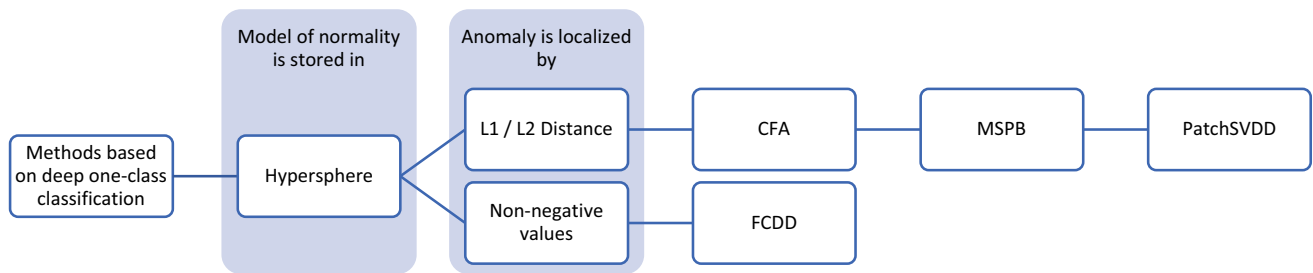


Fig. 7 Relationship and functionality of methods based on deep one-class classification. The models use a hypersphere, sometimes in addition to other techniques, to store their knowledge about nominal images. At inference time, different strategies are used to detect anomalies

4.3 Methods Based on Deep One-Class Classification

Anomaly detection methods typically do not use loss functions explicitly designed for anomaly detection. Deep one-class classification models are inspired by Support Vector Data Description (Tax & Duin, 2004) and are explicitly optimized for the one-class and novelty setting. The adapted loss function allows the neural network parameters to be trained, which is different from, for example, frozen feature extractors or artificially generated anomalies for supervised learning.

For the traditional area of outlier detection, the one-class support vector machine and support vector data description are commonly used algorithms. The methods in this section are inspired by these methods and transfer the ideas to the field of deep learning. The relationship of the methods is shown in Fig. 7.

Fully Convolutional Data Description (FCDD) (Liznerski et al., 2021), belongs to the family of hypersphere classifiers (Ruff et al., 2020a) and follows the Deep Support Vector Data Description (Ruff et al., 2018). The method uses a fully convolutional network, i.e., a network with no fully connected layers. For its optimization, it uses a pseudo-Huber loss, where all operations are applied element-wise:

$$H(x) = \left(\sqrt{f_n(x; \theta)^2 + 1} - 1 \right) \quad (6)$$

$H(x)$ is a matrix on the feature map f_n of f . The FCDD-objective is then defined as

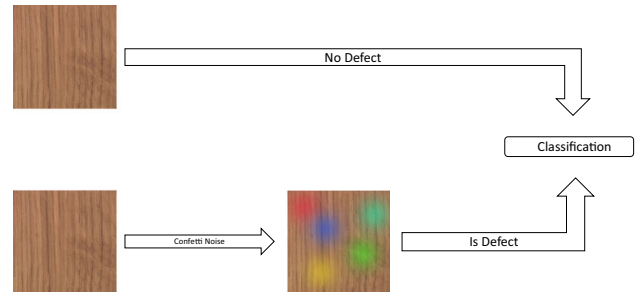


Fig. 8 Example input as used by Liznerski et al. (2021). The classifier learns to detect confetti noise on images, which turns out to be suitable for other types of anomalies as well. The authors show that a small fraction of labeled example (e.g., 5 labeled examples per anomaly) can largely improve the detection results for anomalies

$$\min_{\theta} \frac{1}{|X|} \sum_{x \in X} (1 - y) \frac{1}{u \cdot v} \|H(x)\|_1 - y \log \left(1 - \exp \left(-\frac{1}{u \cdot v} \|H(x)\|_1 \right) \right) \quad (7)$$

where $\|H(x)\|_1$ is the sum of entries in $H(x)$ that are not negative and y denotes whether x belongs to an anomaly. The authors either use arbitrary image data (such as Cifar10 or ImageNet) or artificially created anomalies such as confetti noise (see Fig. 8) to represent outliers. u and v refer to the width and height of the feature map f_n , respectively.

Patch SVDD (Yi & Yoon, 2020) builds on Deep SVDD (Ruff et al., 2018) and extends Deep SVDD with the ability to localize the anomaly in the image. To achieve this, the image is decomposed into individual patches and each patch is mapped to a vector. In Deep SVDD, in the simple case, the vector corresponds to an average of all predictions, which does not work reliably for individual patches. Instead, PatchSVDD replaces the vector of averages with embeddings that map similar image regions to similar embeddings. For learning the embeddings, they rely on solving jigsaw puzzles, as in Doersch et al. (2015).

The ideas behind PatchSVDD (Yi & Yoon, 2020) are incorporated and extended in Tsai et al. (2022) for their method MSPB. Tsai et al. (2022) extract embeddings of dif-

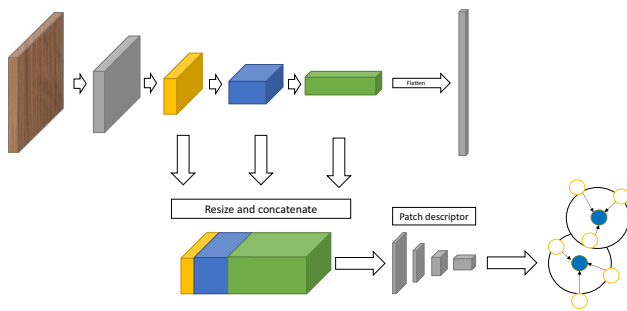


Fig. 9 Schematic overview of CFA (Lee et al., 2022). CFA extracts features of different scales and stores them in a coreset-reduced database. During training, the parameters of the patch descriptor are trained, which maps image patches (yellow circles) onto a hypersphere around memorized training points (blue circles). The hypersphere is defined by k-nearest neighbors, while outliers are projected outside the hypersphere

ferent sizes and perform self-supervised learning on them. For this purpose, they extend the loss of SVDD with additional components. This includes embeddings of neighboring patches to exhibit similarity to each other. The similarity is measured in cosine distance. Another component is the relative position of the patches: analogous to Doersch et al. (2015), the model classifies in which angle two neighboring patches are positioned to each other. The last component of the loss function is based on k-means clustering: the embedding should be as close as possible to the cluster center. The cluster centers are thereby updated every 5 epochs during training.

CFA, proposed by Lee et al. (2022) (Fig. 9), combines a memory bank with metric learning. This is done by first extracting features at different scales and storing them in the database. Then, a function is learned that places the samples within a hypersphere. To make the hypersphere sharper and to identify anomalies more easily, the authors introduced hard-sample mining based on distant neighbors of the point. As with the other parametric models, the advantage of CFA is that the inference is independent of the dataset size.

4.4 Knowledge Distillation/Student Teacher Learning

These methods use two neural networks. The teacher neural network is a pre-trained network. The student neural network learns based on the teacher's prediction, imitating the teacher's behavior. Since the teacher network only teaches images without defects, the student cannot correctly represent images with defects. The difference in prediction between the teacher and the student represents a factor for recognizing the defect.

Knowledge Distillation (Buciluă et al., 2006; Hinton et al., 2015; Gou et al., 2021) aims to transfer the knowl-

edge of one model (teacher) to another model (student). The student-teacher-framework is often used for this purpose. In the student-teacher-framework, the teacher and the student receive the same input, for example, images of products. It is the student's goal to predict the teacher's output. The labels for the student's training come from the teacher's prediction. If the dataset is partially labeled, the labels may additionally be considered when training the student.

In the following, f represents the function of the teacher. The goal is to learn a neural network s with parameters θ_s that imitates the behavior of f :

$$\min_{\theta_s} \frac{1}{|X|} \sum_{x \in X} \mathcal{L}(f(x), s(x; \theta_s)) \quad (8)$$

where $\mathcal{L}$ is a loss function that measures the difference between the predictions of the teacher model and the predictions of the student model. In the context of the methods presented, the teacher's feature map is usually imitated. As these are real values, the mean squared error, for example, is an appropriate loss function $\mathcal{L}$. The teacher usually remains frozen during the training so that the parameters of the network do not change.

The assumption is that the loss for samples that stem from the distribution of X is lower than the loss for samples that do not stem from the distribution of X , i.e., represent outliers. The outlier score s_p per patch results directly from the loss, if evaluated at patch p :

$$s_p = (\mathcal{L}(\theta_s))_p \quad (9)$$

The methods of the previous sections use, for example, a database or a parametric model like the Gaussian distribution to match the current image with past images. All variants aim to have a model of a normal image. With the notion of the normal image, it is easier to detect anomalies in future images.

In the student-teacher-framework, the student takes the place of the database or the Gaussian distribution. During the training, the student only experiences images that do not contain defects. This means that the student builds the model of how normal images without defects look like. Later, defects can be identified by comparing the output of the student and teacher: since the student knows only the concept of normal images, its prediction of defects differs from the teacher's prediction. The difference between the student's and the teacher's prediction then forms a measure for detecting the defects. This is shown schematically in Fig. 10. The relationship of the methods is shown in Fig. 11.

Concretely, the approach for anomaly detection is implemented in the work of Bergmann et al. (2020). The authors work with two models that, by design, see only one patch of the image at a time. The models convolve over the entire

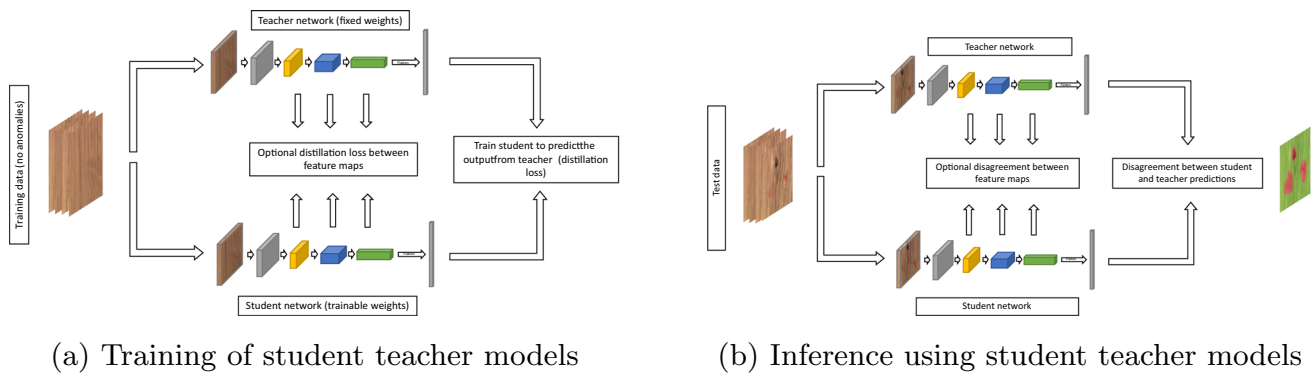


Fig. 10 Training and inference using student teacher models

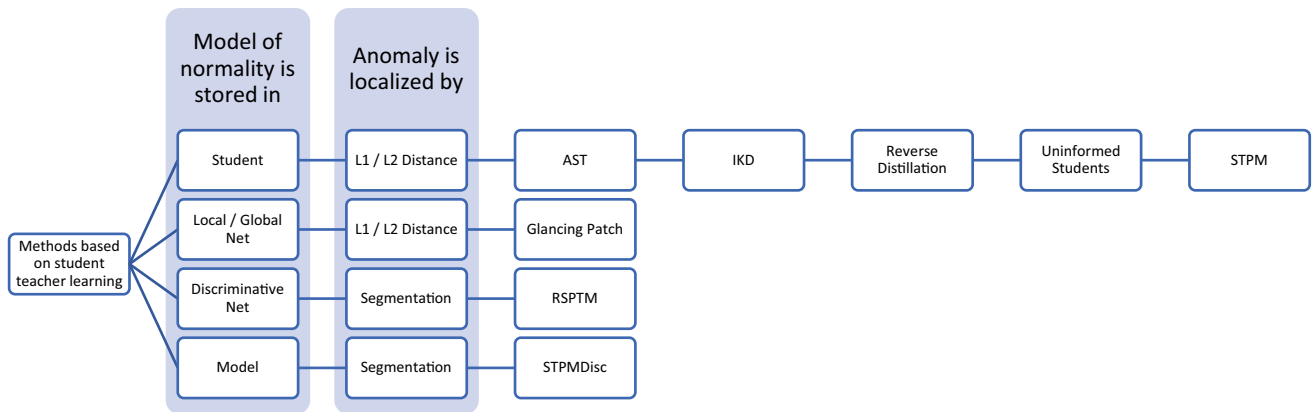


Fig. 11 The relationship and functionality of methods based on student teacher learning. Usually, knowledge about normal images is stored in the student's network. However, there are variants as well. There are also different strategies at inference time, while anomaly detection based on distances is also common

image to produce a dense embedding for each patch of the image. For training, the teacher provides an embedding for the patch of the image, and the student attempts to match the prediction. During training, the following loss is minimized:

$$\mathcal{L}(\theta_s) = \frac{1}{|X|} \sum_{x \in X} \|s(x; \theta_s) - f(x)\|^2 \quad (10)$$

In addition, the authors also add a compactness loss $\mathcal{L}_c(s(x)) = \sum_{i \neq j} c_{ij}$ to their final loss function. c_{ij} denotes the entries of the correlation matrix that is computed over all the descriptors $s(x)$ in the current mini-batch. Since the models each work on a patch of the image, localization of the defect is possible. The defects are detected by observing the prediction difference between student and teacher, as well as the intrinsic model uncertainty (Kendall & Gal, 2017) of the student.

The teacher in this approach is a pre-trained model on arbitrary data, such as ImageNet. One challenge of the method is that the image patch is comparatively small. Pre-trained models on ImageNet typically have larger images as input. The authors therefore show alternative ways of learning a teacher

model to gain the embedding of the patch. For this, they use patches based on ImageNet and metric learning as a type of supervised learning method. Positive patches are obtained by neighboring image regions, while negative patches are taken from other images.

A related work to Bergmann et al. (2020) is the work of Wang et al. (2021a) (STPM). In Wang et al. (2021a), the authors use several feature maps (denoted as $F = \{1, 2, 3\}$) of the teacher for the training of the student. This allows for a more accurate localization of the anomaly, as used for methods based on pre-trained models. The score for detecting the defect is given by the difference of each feature map between teacher and student. By resizing the feature maps, the defect can be projected on the entire image size. During training, for every feature map $i \in F$ the method minimizes:

$$\mathcal{L}(\theta_s) = \frac{1}{|X|} \sum_{x \in X} \sum_{i \in F} \left\| \frac{s_i(x; \theta_s)}{\|s_i(x; \theta_s)\|_2} - \frac{f_i(x)}{\|f_i(x)\|_2} \right\|^2 \quad (11)$$

Since the student and teacher predictions are L2-normalized ($\frac{f_i(x)}{\|f_i(x)\|_2}$), effectively the Euclidean distance is minimized. An extension to STPM is proposed in Yamada et al. (2022)

by training a student to reconstruct teacher feature maps. The authors also use synthetic anomalies based on DRÆM (Zavrtanik et al., 2021) to improve the anomaly detection rate.

The work of Wang et al. (2021b) also relies on knowledge distillation, but checks whether a local patch matches the global embedding of the image. Two neural networks are used for this purpose. The first network predicts the global embedding of the image, the second network predicts a local embedding for each patch of the image. Afterward, it is checked whether the local embedding matches the global embedding. For this purpose, the Euclidean distance between the local and the global embedding is computed. Another component of the approach is the Distortion Anomaly Detection Head. This is a second output of the network, which classifies whether a defect is visible on the local image section. This part of the network is trained on artificially generated anomalies by inserting a random stain on the image section. The Distortion Anomaly Detection Head is trained to check whether a stain is visible on the image or not. The anomaly score of the method is composed of the Euclidean distance between local and global embedding and the probability of the Distortion Head that an anomaly is visible on the image.

Deng and Li (2022) (Fig. 12) uses a different input for the student-teacher model. Instead of the RGB input, the student receives the embedding of the image and has to reconstruct the intermediate representations of the teacher. This *reverse distillation* approach has much in common with an encoder-decoder approach in terms of architecture. During training, the cosine similarity for every feature map $i \in F$ and every pixel $p \in P_i$ in the feature map i is maximized:

$$\mathcal{L}(\theta_s) = \frac{1}{|X|} \sum_{x \in X} \sum_{i \in F} \frac{1}{|P_i|} \sum_{p \in P_i} \left(1 - \frac{s_i(x; \theta_s)_p^T \cdot f_i(x)_p}{\|s_i(x; \theta_s)_p\|_2 \cdot \|f_i(x)_p\|_2} \right) \quad (12)$$

The work of Cao et al. (2022) suggests that student overfitting can occur because the capacity of the network can easily capture the available signal. For this reason, they propose a method called Informative Knowledge Distillation (IKD). IKD combines the mean squared error of the loss function between student and teacher with a cosine similarity between the features. Moreover, the authors add a hard-sampling-mining to give more attention to such images that are particularly relevant to the student's learning process.

Besides the usual setup of having one student and one teacher, Yamada and Hotta (2021) propose a variant to this. Their method Student-Teacher Feature Pyramid Matching (STPM) uses two students and two teachers. One pair of student-teacher follows the ordinary strategy of knowledge

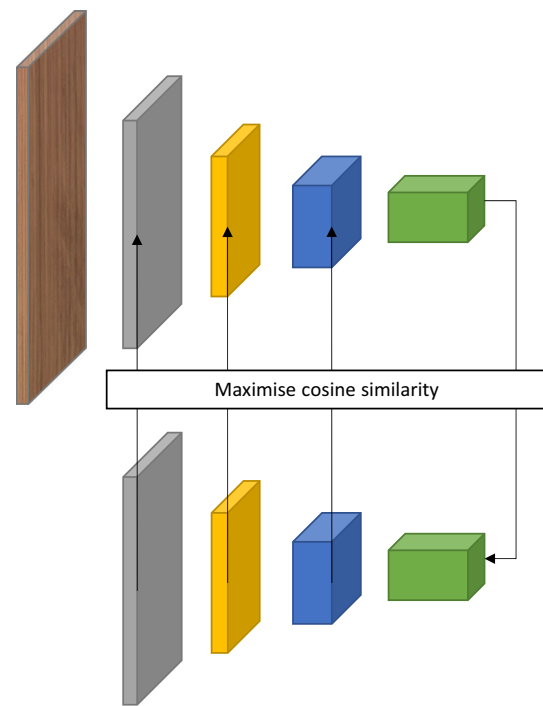


Fig. 12 The data flow for reverse distillation (Deng & Li, 2022). Unlike other methods, the student (bottom) must reconstruct the teacher's features (top) based on the last feature map f_n . The loss function used for learning is the vector-wise cosine similarity for every pixel in the feature map

distillation and discrepancy between student teacher predictions. The second pair of student-teacher is trained to reconstruct the pixel input based on the first teacher. This reconstruction-based approach improves the localization and detection of the anomalies compared to the simple student-teacher approach.

Typically, in student-teacher learning, the same neural network architecture is chosen for the student and the teacher. Rudolph et al. (2022) find that the differences between student and teacher, which are crucial for the detection of defects, are often small. Instead, they introduce a different net architecture (in this case based on Normalizing Flows) for the teacher than for the student (in their work a convolutional net). For the teacher, they also use positional encodings, analogous to CFlow-AD (Gudovskiy et al., 2021).

4.5 Methods Based on Artificially Created Anomalies

These methods use supervised learning to detect the defects. The special feature is that the labels for learning are not annotated by humans, but generated by the computer. This can be done, for example, by inserting artificial anomalies into the image. In this way, supervised learning methods can be used without requiring expensive human annotation.

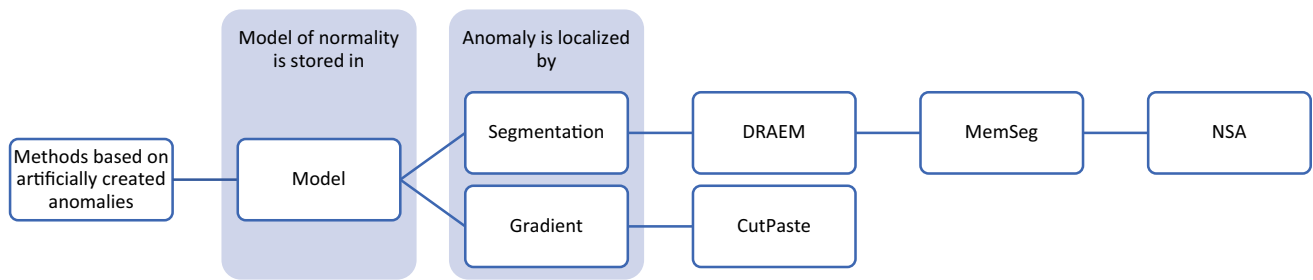


Fig. 13 The relationship and functioning of methods based on artificially generated anomalies. Since the methods use supervised learning to detect anomalies, they have implicit knowledge in the model about

how to detect anomalies. At inference time, the representations of the artificial anomalies are sufficient to detect real anomalies

For student-teacher learning, the teacher provides the label for the student. This allows learning based on labels without the need for human labeling. Self-Supervised Learning also aims to use supervised learning methods without using human-generated labels. A simple way to automatically generate labels is to rotate an image (0° , 90° , 180° , 270°). Then a model is trained that classifies by how many degrees the image was rotated. This approach was proposed in Gidaris et al. (2018). Other works cut out adjacent patches from the image and let a network learn the correct composition of the patches. This solving of the jigsaw puzzle was proposed in Noroozi and Favaro (2016). More recently, contrastive methods have been pursued by the community to learn self-supervised. An example of contrastive methods is to take an image and create two different versions of the image by transforming the image. Subsequently, a neural network learns to map the different versions of the image to an embedding that is as similar as possible, regardless of the transformation. An overview of contrastive learning methods can be found in Jaiswal et al. (2021a).

Self-supervised learning trains the model without costly annotations. This is a pre-training to reduce the number of labels needed for the actual task. After self-supervised learning, a portion of the network is usually removed to solve the actual task. This is similar to traditional transfer learning. Nevertheless, it is necessary to have a labeled dataset at the end, which provides labels to the task to be solved.

For unsupervised defect detection, there are no human-annotated labels that can be used after the self-supervised task. For this reason, ways must be found to use self-supervised learning for defect detection without the presence of a human annotated dataset.

A work that rotates images and then detects anomalies based on this model was presented by Golan and El-Yaniv (2018). The work uses a variety of geometric transformations on the image to create a model that can distinguish normal images from transformed images. At training time, the model learns which transformation was applied to the image. At inference time, the model's prediction of whether a transfor-

mation was applied to the image is used. The authors show that this method can detect anomalies when the anomalies are semantically very different from the normal images.

For detection of subtle anomalies and defects in manufactured pieces, detection based on rotation is insufficient (Li et al., 2021). In the community, synthetic anomalies are therefore added to the dataset, which are useful as labels for anomaly detection. It has been found that detection based on the artificial anomalies can reliably generalize to real anomalies. The relationship of the methods is shown in Fig. 13.

A work that generates labels based on synthetic anomalies is CutPaste (Li et al., 2021). For this, the authors cut out a small patch from the image, rotate this patch, optionally rotate pixels in the patch, and paste the patch at an arbitrary, different location. Once an image has been anomalized in this manner, a model can learn whether a patch has been altered on the image. Labels generated in this way are suitable for anomaly detection without the need for additional human annotation. The method is shown schematically in Fig. 14.

Let f be the neural network that learns the defect detection. Since the dataset X does not contain any anomalies, an additional function $d(x)$ is introduced that inserts artificial anomalies on x . For CutPaste, d refers to the CutPaste operation. If d is applied on x , an anomaly is visible on the image. During training, CutPaste minimizes cross-entropy:

$$\begin{aligned} \mathcal{L}(\theta_f) &= \frac{1}{|X|} \sum_{x \in X} \text{CE}(f(x; \theta_f), 0) + \text{CE}(f(d(x); \theta_f), 1) \end{aligned} \quad (13)$$

The work of Schlüter et al. (2021) (NSA) uses a similar approach. They argue that CutPaste's anomalies are too synthetic and therefore the risk of overfitting the model is high. Consequently, Schlüter et al. (2021) proposes an improved method to generate synthetic anomalies. For this, they make use of Poisson Image Editing, which inserts the anomalies into the image more naturally than CutPaste does. The method is shown schematically in Fig. 15. Depending on the

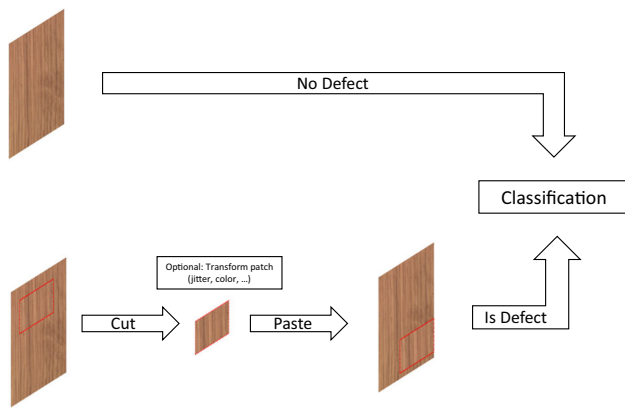


Fig. 14 Example input for CutPaste (Li et al., 2021) based anomaly detection. The models learn the difference between images in which a patch has been modified with CutPaste and those in which this is not the case

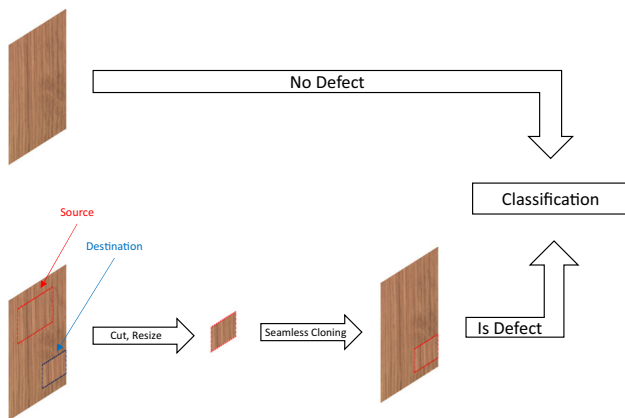


Fig. 15 Example input for NSA-based anomaly detection as proposed by Schlüter et al. (2021). The approach is similar to CutPaste. However, the crafted anomalies resemble more to natural anomalies. As in CutPaste, the model is trained to detect synthetic patches in the image

setting of NSA, either cross-entropy or mean-squared error is used as the loss function.

The work by Zavrtanik et al. (2021) proposes DRÆM. DRÆM uses two autoencoders to discover the real defects in the images based on artificial defects. For this purpose, in the first step, anomalies are inserted into the image, which an autoencoder subsequently removes (reconstruction network). The second autoencoder receives both the reconstructed variant and the original to localize the defects in the image (discriminative network).

MemSeg Yang et al. (2022) (Fig. 16) uses a U-Net like structure to learn a segmentation of the image. For this purpose, they introduce artificial anomalies and can thus segment the anomalies on the image, since the ground truth is known. To support the learning process, they also work with a memory bank. The information from the memory bank is provided to the model via feature fusion and spatial attention. During

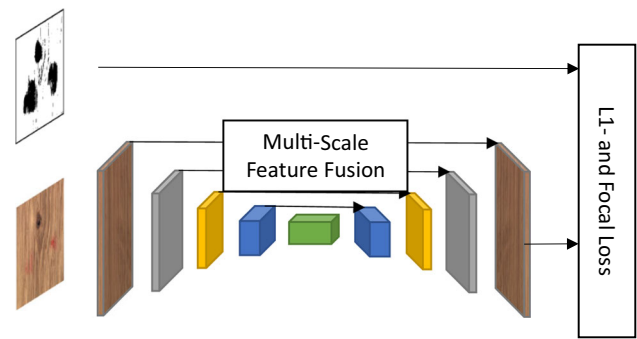


Fig. 16 The schematic overview of MemSeg (Yang et al., 2022). As input, MemSeg generates synthetic anomalies, which provides a ground truth for the supervised segmentation of the image. The U-Net-like model is optimized via L1 and focal loss. A distinctive feature is the Multi Feature Fusion module, which extracts features from different scales and makes them available during up-sampling via attention mechanisms

training, MemSeg minimizes the sum of L1:

$$\mathcal{L}_{L1}(\theta_f) = \|y - f(x; \theta_f)\|_1 \quad (14)$$

and Focal loss:

$$\mathcal{L}_{Focal}(\theta_f) = -\alpha_t (1 - p_t)^\gamma \log(p_t) \quad (15)$$

where p_t refers to the predicted probability p of the pixel category when at p an anomaly is present and $p_t = 1 - p$ if no anomaly is present. α_t and γ are hyperparameters to control the shape of the loss curve (γ) and weighting regarding class imbalance (α_t). MemSeg balances both loss terms for accurate detection of anomalies:

$$\mathcal{L}(\theta_f) = \lambda \mathcal{L}_{L1} + (1 - \lambda) \mathcal{L}_{Focal} \quad (16)$$

4.6 Reconstruction-Based Methods

Reconstruction-based methods reduce the dimension of the image to a much smaller vector in a first step. Then, they reconstruct the complete image from the small vector. When comparing the original image and the reconstructed variant, differences arise. Due to the information leakage, defects are reconstructed worse than images without defects. The difference between the original image and the reconstruction is a way to detect defects in the image.

Another family of methods relies on the reconstruction of data. This involves taking an image, reducing the dimension of the image, and recovering the full dimension from the reduced dimension (see Fig. 17 and Fig. 18). The reduced dimension in the middle represents a bottleneck for the information, so that not every detail can be recovered afterward. In particular, it turns out that anomalies can be recovered

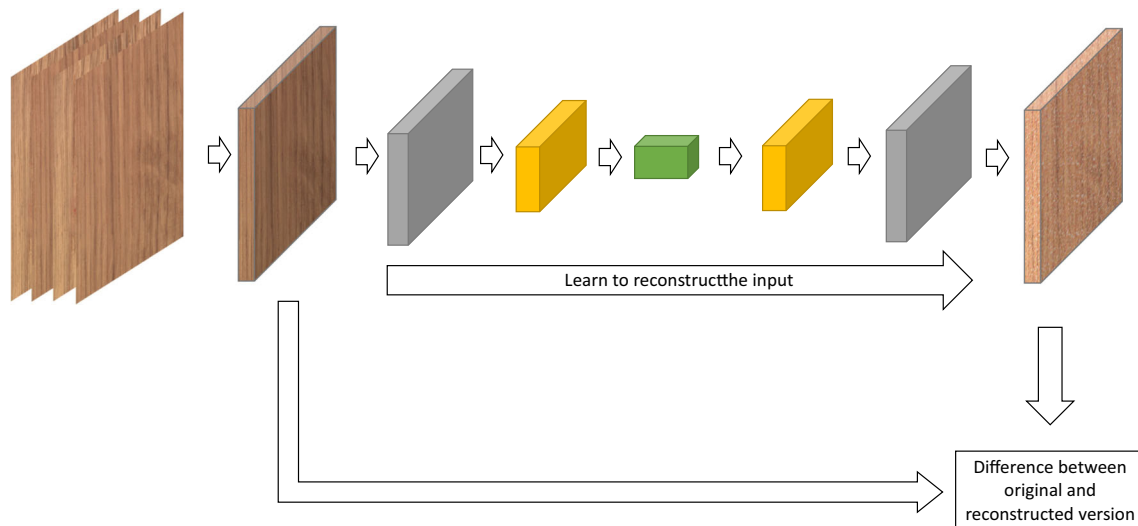


Fig. 17 Schema for methods based on reconstruction of input data

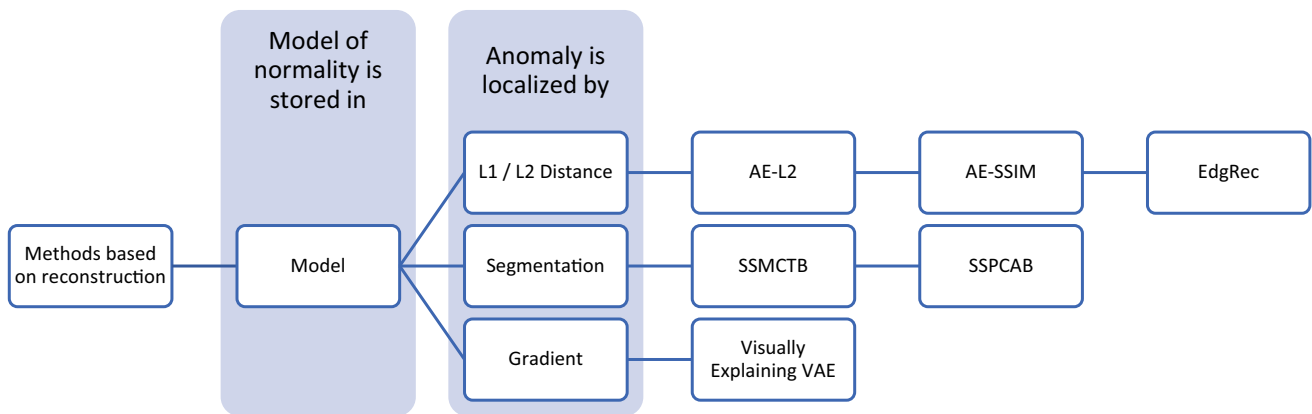


Fig. 18 Relationship and functioning of methods based on reconstruction. Since the models are trained only on images without anomalies, the information for reconstructing anomalies insufficiently passes through the bottleneck of the models. Knowledge about nominal images is there-

fore stored in the model. At inference time, there are different strategies for detecting anomalies, with distances between input and output being a common approach

worse from the low dimension than it is the case for nominal images. The difference between the original image and the recovered variant is comparatively large for anomalies. This is also a natural measure for anomaly detection. Results on the MVTec dataset using this type of autoencoder were presented in Bergmann et al. (2021a). Let e and d be the encoder and decoder, respectively. During training, losses such as mean squared error:

$$\min_{\theta_e, \theta_d} \frac{1}{|X|} \sum_{x \in X} (x - d(e(x; \theta_e); \theta_d))^2 \quad (17)$$

or L1 distance are minimized. For binary input data, cross-entropy loss is also a valid choice. The outlier score s_p per

pixel results directly from the loss, if evaluated at pixel p :

$$s_p = (\mathcal{L}(\theta_e; \theta_d))_p \quad (18)$$

For inconspicuous and subtle anomalies, such as those considered in this survey, the conventional approach of dimension reduction via ordinary autoencoders is not reliable. Inaccurate recovery of edges on images or minor visual changes result in recovery of these areas being so inaccurate that the actual anomalies cannot be detected (Bergmann et al., 2018).

For this reason, alternatives to conventional autoencoders specialized in the detection of the defects have been proposed. The work of Bergmann et al. (2018, 2021a) proposes to modify the loss function of the autoencoder. For this purpose, they use a function that considers structural similar-

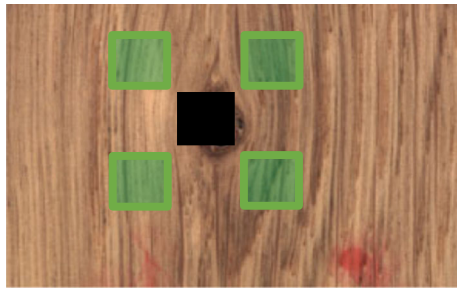


Fig. 19 The schematic overview of SSPCAB (Ristea et al., 2021). SSPCAB extends existing models with the masked convolutional kernel. In this kernel, the network only has access to the areas marked in green, while the black area in the middle is masked. It is the task of the network to reconstruct the surface in the middle

ities of different image regions. This is done by comparing brightness, contrast, and structural information, instead of the simple reconstruction error of the image. Brightness is estimated using the mean of the patch, contrast corresponds to the variance of the patch, and structural information is obtained using the covariance of two patches. This definition corresponds to the proposal of Wang et al. (2004).

Autoencoders are widely used for anomaly detection. To further improve the explainability of the results, Liu et al. (2019) have proposed a method that further improves the localization of anomalies. They rely on gradient-based explanations of models, which, however, require a classification or regression task. In the case of autoencoders, neither is available. The authors fill this gap and propose a method to provide a gradient-based explanation of autoencoder results. They show that their method performs better on the MVTec dataset than the standard variant of autoencoders.

Ristea et al. (2021) propose an extension based on attention modules. The proposed Self-Supervised Predictive Convolutional Attentive Block (SSPCAB) does not represent an own method, but can be integrated with existing methods to improve their performance. SSPCAB consists of a convolutional block where the center of the receptive field is masked. The self-supervised task of the block is to recover the masked center. Attention mechanisms are used for this purpose. An L2 loss is used for reconstruction in the SSPCAB block:

$$\mathcal{L}(\theta_g) = (g(x) - x)^2 \quad (19)$$

where g stands for the SSPCAB block as presented in Fig. 19. Since SSPCAB can be combined with any learning algorithm, the total loss is the sum of the original loss function $\mathcal{L}(\theta_f)$ and the loss from SSPCAB:

$$\mathcal{L}(\theta_f, \theta_g) = \mathcal{L}(\theta_f) + \lambda \cdot \mathcal{L}(\theta_g) \quad (20)$$

with λ as hyperparameter, which balances the relevance of reconstructing the masked pixels to the task of f . The authors

use DRoEM and CutPaste to show that their extension can further improve the performance of the methods. Improvements for SSPCAB were presented in Madan et al. (2022) by replacing 2D masked convolutions with 3D masked convolutions. The authors also find that channel-wise attention further improves performance.

One difficulty for reconstruction-based approaches is that frequent and changing regions of the image can lead to false negatives in recognition because they can be poorly learned by the models. Liu et al. (2022) therefore propose to learn a reconstruction of the input based only on gray value edges. For this purpose, they use a U-Net like denoising autoencoder. Converting to gray value edges removes unsteady areas of the image and thus avoids high reconstruction errors due to changing areas in the image.

4.7 Generative Adversarial Networks

Generative Adversarial Networks consist of two neural networks, the generator and the discriminator. During training, the generator generates images, while the discriminator has to decide which image was generated by the generator and which image comes from the data set. The discriminator is therefore a way of detecting images that do not match the data set and may represent outliers.

Generative Adversarial Networks (GANs) consist of two neural networks, the generator and the discriminator. During training, the generator generates images. The discriminator has to decide which image is generated by the generator and which is from the data set. Images that do not match the data set and may be outliers are therefore detected by the discriminator. Over time, the generator gets better at producing photorealistic images, which also improves the discriminator's ability to detect outliers. The relationship of the methods is shown in Fig. 20.

Liang et al. (2022) rely on GANs for their method OCR-GAN. OCR-GAN consists of several generators which recover the original image from frequency decoupled images.

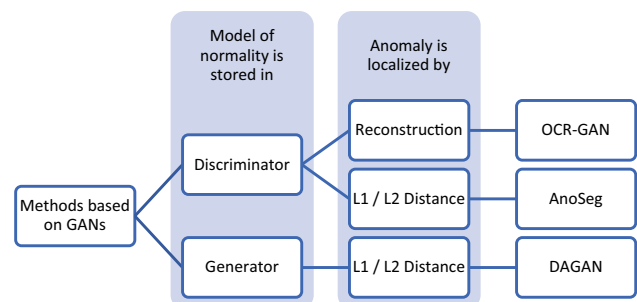


Fig. 20 Relationship and functioning of methods based on GANs. The knowledge about nominal images is usually stored in the discriminator, while the detection of anomalies at inference time can be achieved using distances, among other things

For frequency decoupling, the images are blurred in iterations with a Gaussian kernel and then the difference from the previous variant is stored. Each of the generators learns the reconstruction of the respective frequency decoupled image. At the end, the individual reconstructions are summed to train the discriminator of the GAN. In this way, end-to-end learning is possible.

Tang et al. (2020) also rely on GANs for their method DAGAN. DAGAN combines GANs with autoencoders, showing that the combination of autoencoders and GANs allows for better anomaly detection rates than it is the case with GANs alone. The authors' observation holds especially when only limited data is available, which is a common problem in defect detection.

The work AnoSeg by Song et al. (2021) uses CutPaste in a modified form to insert patches of heavily transformed images into the original image. The authors also use additional input in the form of spatial information about the pixel by specifying a map of coordinates in addition to the pixel values, as proposed in CoordConv (Liu et al., 2018). The coordinates allow for improved spatial awareness of the neural network when detecting the inserted patch.

4.8 Normalizing Flows

A Normalizing Flow is a function which transforms a complex probability distribution (e.g., the images of MVTEC AD) into a simple distribution (e.g., Gaussian distribution). The function is bijective and differentiable, which means that each point of the simple distribution corresponds to a point in the complex distribution. By measuring the density of the simple distribution, the probability for the point in the complex distribution can be determined. Normalizing Flows provide a way to directly measure the probability of occurrence of an image, which corresponds to the natural definition of outliers.

A natural definition of an anomaly is that it occurs rarely. In probability terms, this means that the anomalous point has a low density of the associated probability distribution. If the probability distribution is known, it is easy to identify outliers.

The problem with anomaly detection is that the probability distribution can be unknown and very complex. The images of the MVTEC anomaly dataset follow a certain probability distribution, but the distribution is unknown.

Normalizing Flows are functions that map complex probability distributions (such as the images of the MVTEC dataset) to a simple distribution (such as a multivariate Gaussian distribution). The functions are bijective, meaning that each point in the simple distribution corresponds to a point in the complex distribution. The probability of a point in the complex distribution can then be determined using the simple

distribution and scaled to the complex distribution (Kobyzev et al., 2021).

In other words, Normalizing Flows represent a family of models that allow an explicit estimation of the density in complex distributions and are thus suitable for anomaly detection. This allows the identification of points that have a low probability of occurrence. A threshold defines the binary decision which points represent an anomaly and which do not. This is shown schematically in Fig. 21. The relationship of the methods is shown in Fig. 22.

DifferNet (Rudolph et al., 2020) uses Normalizing Flows to identify anomalies on MVTEC AD. The method is similar to feature extraction methods such as PaDiM, SPADE or PatchCore and also uses a pre-trained network to extract features. Instead of databases or k-nearest neighbors, DifferNet uses the Normalizing Flow to map the extracted features of the image to the Gaussian distribution. Analogous to the feature extraction methods, DifferNet uses several intermediate layers of the pre-trained network.

The anomaly score s for the entire image is calculated using different transformations $t \in \mathcal{T}$ of the image:

$$s(x) = \frac{1}{|\mathcal{X}|} \sum_{t \in \mathcal{T}} [-\log p_Z(f_{\text{NF}}(f_n(t(x))))] \quad (21)$$

with f_{NF} being the Normalizing Flow and f_n is the feature extractor. Due to its architecture, DifferNet cannot immediately locate the errors. For this reason, the authors use gradient-based anomaly localization to determine which pixels have a particular influence on the log likelihood and can be marked as anomalous:

$$g(x) = \sum_{c \in \mathcal{C}} |G * \nabla x_c| \quad (22)$$

The gradient map $g(x)$ sums over all channels $c \in \mathcal{C}$ and smoothes the gradients with the Gaussian filter G for better visualization.

CFLOW-AD (Gudovskiy et al., 2021) builds on DifferNet and follows the same strategy as DifferNet for feature extraction. In an identical way, features are extracted from a pre-trained network at multiple scales. The difference is that DifferNet does not encode information from which spatial orientation the feature vector was extracted. CFLOW-AD additionally conditions the extracted feature vector with a spatial indication to ensure better localization of the anomaly.

The challenge to obtain the spatial coordinate of the feature vector arises when flattening the features. Flattening is necessary because many methods for Normalizing Flows are implemented based on Fully Connected Networks. Fast Flow (Yu et al., 2021) points out this difficulty and replaces the Normalizing Flow with fully convolutional networks with 3×3 convolutions followed by 1×1 convolutions. In this way,

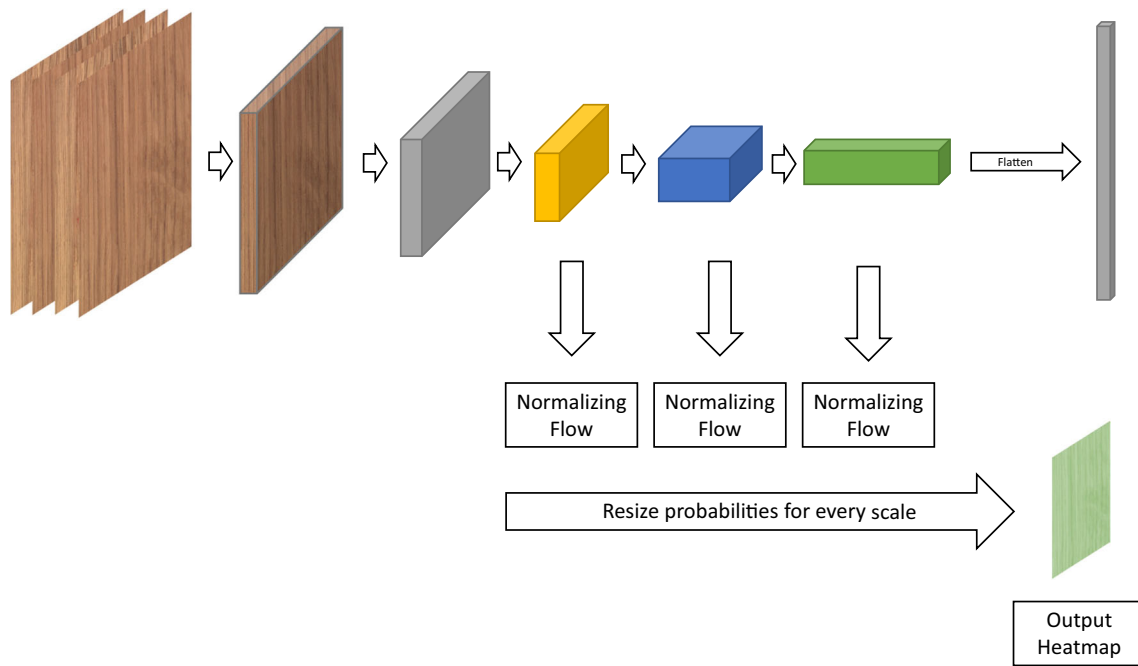


Fig. 21 Schema for methods based on Normalizing Flows

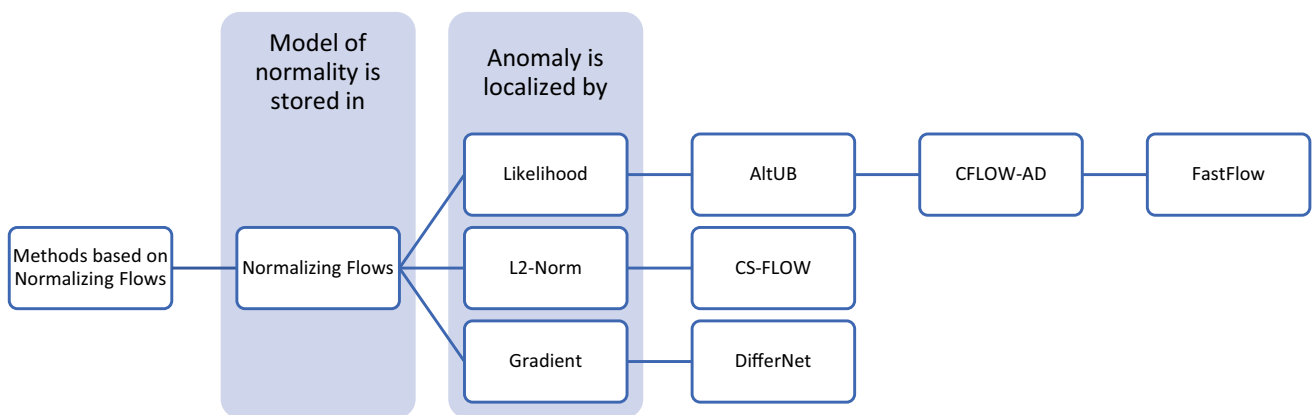


Fig. 22 Relationship and functioning of methods based on Normalizing Flows. The knowledge about nominal images is stored in the Normalizing Flow, while different strategies are chosen to detect anomalies at inference time

no feature flattening is necessary and the spatial component of the features is not lost.

CS-FLOW (Rudolph et al., 2021) is an extension of DifferNet and works similar to FastFlow based on fully convolutional networks. The processing of feature maps at different scales is done simultaneously, hence the name Cross-Scale (CS) Flow. Furthermore, the authors argue that the assumption of normal distribution of the extracted features is not true in practice, as it is necessary for example to estimate the Mahalanobis distance.

While normalizing flows are designed to map the images to a predetermined probability distribution (e.g., $\mathcal{N}(0, I)$), Kim et al. (2022) note that this is often not the case. They also highlight that training Normalizing Flows can be unsta-

ble, which is particularly problematic in the unsupervised setting, where the instability may not be detectable. For this reason, they propose AltUB, an alternative training method that allows for better mapping of the probability distribution and stabilizes the training of Normalizing Flows.

Unlike the other methods, AltUB allows learning not only the parameters of the Normalizing Flow, but also μ and Σ of the base distribution. To achieve this, they use a higher learning rate for the parameters of the base distribution, but only update these parameters at a fixed interval. This is shown schematically in Fig. 23.⁸

⁸ In version 2 of the PatchCore paper, the authors note that an ensemble of different backbones for extracting the embeddings raises the AUC

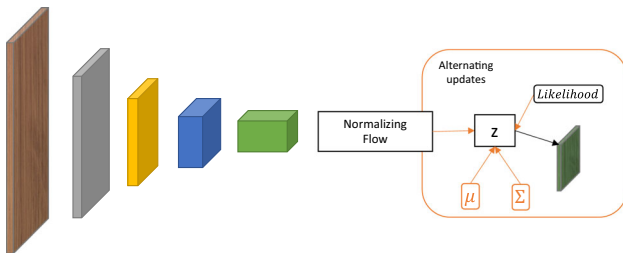


Fig. 23 The schematic overview of AltUB (Kim et al., 2022). The Normalizing Flow, μ and Σ of the distribution are trainable variables. The orange arrows represent the alternating updates

5 Benchmarks and Comparison

5.1 Benchmarks for Detection and Segmentation

We rank the models by the average Area Under the Receiver Operating Characteristic Curve (AUC) at the image level (detection) and at the pixel level (segmentation). If reported by the methods, the PRO metric (segmentation) is also noted. The results for detection are shown in Table 3a, and the results for segmentation are shown in Table 3b.

For detection (Table 3a), MemSeg is the best method according to this metric. MemSeg achieves an average AUC of 99.56 across all 15 classes of the MVTec AD dataset. CFA ranks as the second-best method, with an average AUC of 99.53. PNI ranks third in the benchmark, AUC is 99.52.

MemSeg and CFA show similar behavior on the different classes. In particular, Leather and Hazelnut are not difficult for PNI, although these classes are among the most difficult for other methods. This suggests that PNI's neighborhood information pays off (Fig. 24a). A note on PatchCore: In an updated version of PatchCore, the authors use an ensemble of backbones to achieve an AUC of 99.6. This makes inference slower, but is the overall very best score of any previous method for this task. The extension of DRÆM by the Attention Block (DRÆM+SSPCAB / DRÆM+SSMCTB) pays off: Compared to the base variant DRÆM, the method achieves about one point more in AUC.

The segmentation (Table 3b) is led by PNI with an average AUC of 98.91 at pixel level. MemSeg is in second place with 0.06 points difference in AUC. FastFlow+AltUB is close behind in third place. N-pad takes the fourth place (98.75 AUC). Special attention should also be paid to the PRO score: while Glancing Patch is in the lower range when measured by AUC, it is the top method when measured by PRO score. This highlights the need for future methods to include multiple metrics when reporting performance. AUC alone is not enough.

value to 99.6. This is the best score of all methods. Since no performance per class was reported in version 2, we fall back to the version 1 values and note the AUC value of 99.1.

Some papers report the performance of their method for both the detection and segmentation tasks. Based on the rank, it can be determined which of the methods is best for the combination of both tasks. The ranking is obtained by taking the average of the rank for detection and the rank for segmentation. In this ranking, MemSeg is the best method, followed by PNI, N-pad and CFA. The results are shown in Table 4.

The MVTec AD consists of 15 different classes. Detecting the defects in the images varies in difficulty and varies for different methods. For detection, the screw is the easiest class across all methods, while the bottle contains the most difficult defects. However, the difficulty varies: PNI has greater difficulty with screws, while it detects leather or hazelnut with relative ease. See the Fig. 24a.

For the difficulty in the task of segmentation, leather leads the list of the most difficult defects. Wood, transistor, tile, and cable are comparatively easy for the methods to detect. There are differences here as well. MemSeg has great difficulty with tile, but relatively little difficulty on screw. See the Fig. 24b.

When looking at the average AUC values, it is more difficult to detect differences in performance. When looking at the box plots, in contrast, the spread of results among the methods is immediately apparent: while some methods consistently achieve high scores across all categories, other methods exhibit a broad spread in AUC. The results for detection and segmentation are shown in Figs. 25a and 25b.

5.2 Benchmark for Inference Speed

The detection rate of anomalies at the image and pixel level does not tell the whole story. For practical applications, the time taken by the methods to make a decision also makes a significant difference. For example, FAPM can process up to 46 images per second (FPS). This allows video analysis to be carried out in real time. PatchCore, on the other hand, achieves 6 FPS in its simplest version with comparable performance. For FastFlow (Yu et al., 2021) it also becomes clear which component of the model can provide fast inference times: Depending on the choice of backbone, 30.8 FPS are achieved (ResNet18) or even only 3.08 with a highly parameterized transformer backbone such as CaiT (Touvron et al., 2021).

For PaDiM (Defard et al., 2021) it is also the backbone that takes a lot of time. When switching from WideResNet50 to ResNet18, 5 instead of 1 FPS can be achieved, while the loss in AUC remains small. CFLOW-AD (Gudovskiy et al., 2021) achieves 32 FPS with the same backbone (WideResNet50), which is also sufficient for real-time analysis.

The inference time as well as the hardware requirements for running the models thus depend significantly on the backbone. Yu et al. (2021) show that the additional inference time can be reduced by using parameterized models to locate the anomaly. However, this also increases the size of the mod-

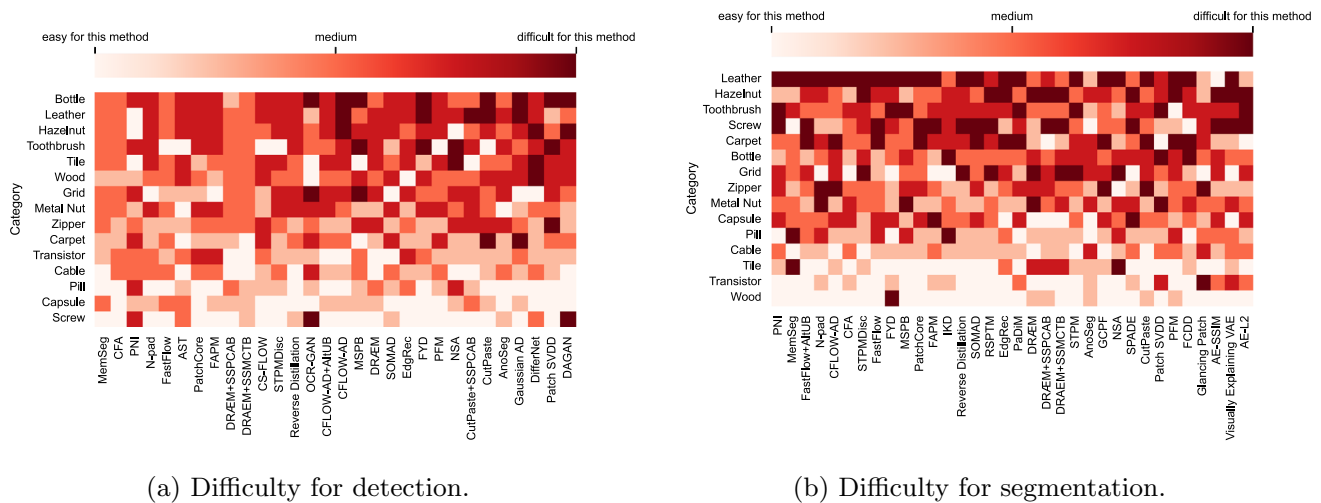
Table 3 Area Under the Receiver Operating Characteristic Curve (AUC) values for every method for the detection and segmentation task

(a) Results for detection		Method	AUC on image level
1		MemSeg (Yang et al., 2022)	99.56
2		CFA (Lee et al., 2022)	99.53
3		PNI (Bae et al., 2022)	99.52
4		N-pad (Jang et al., 2022)	99.38
5		FastFlow (Yu et al., 2021)	99.37
6		AST (Rudolph et al., 2022)	99.14
7		PatchCore (Roth et al., 2021)	99.10
8		FAPM (Kim et al., 2022)	8.99
9		DRÆM+SSPCAB (Ristea et al., 2021)	98.86
10		DRÆM+SSMCTB (Madan et al., 2022)	98.75
11		CS-FLOW (Rudolph et al., 2021)	98.72
12		STPMDisc (Yamada et al., 2022)	98.71
13		Reverse Distillation (Deng & Li, 2022)	98.46
14		OCR-GAN (Liang et al., 2022)	98.30
15		CFLOW-AD+AIUB (Kim et al., 2022)	98.29
16		CFLOW-AD (Gudovskiy et al., 2021)	98.26
17		MSPB (Tsai et al., 2022)	98.15
18		DRÆM (Zavrtanik et al., 2021)	97.98
19		SOMAD (Li et al., 2021)	97.93
20		EdgRec (Liu et al., 2022)	97.81
21		FYD (Zheng et al., 2021)	97.65
22		PFM (Wan et al., 2022b)	97.53
23		NSA (Schlüter et al., 2021)	97.20
24		CutPaste+SSPCAB (Ristea et al., 2021)	96.91
25		CutPaste (Li et al., 2021)	96.65
26		AnoSeg (Song et al., 2021)	96.13
27		Gaussian AD (Rippel et al., 2020)	95.81
28		DifferNet (Rudolph et al., 2020)	94.69
29		Patch SVDD (Yi & Yoon, 2020)	92.07
30		DAGAN (Tang et al., 2020)	87.32

Table 3 continued

(a) Results for detection		Method		AUC on image level	
(b) Results for segmentation		Method	AUC on pixel level	PRO-Score	
1		PNI (Bae et al., 2022)	98.91	–	–
2		MemSeg (Yang et al., 2022)	98.85	–	–
3		FastFlow+AIUB (Kim et al., 2022)	98.85	–	–
4		N-pad (Jang et al., 2022)	98.75	95.10	95.10
5		CFLOW-AD (Gudovskiy et al., 2021)	98.62	94.60	94.60
6		CFA (Lee et al., 2022)	98.47	83.20	83.20
7		STPMDisC (Yamada et al., 2022)	98.47	95.10	95.10
8		FastFlow (Yu et al., 2021)	98.45	–	–
9		FYD (Zheng et al., 2021)	98.23	93.00	93.00
10		MSPB (Tsai et al., 2022)	98.14	95.50	95.50
11		PatchCore (Roth et al., 2021)	98.07	93.50	93.50
12		FAPM (Kim et al., 2022)	97.99	–	–
13		IKD (Cao et al., 2022)	97.81	92.55	92.55
14		Reverse Distillation (Deng & Li, 2022)	97.81	93.90	93.90
15		SOMAD (Li et al., 2021)	97.79	93.30	93.30
16		RSPTM (Yamada & Hotta, 2021)	97.71	–	–
17		EdgRec (Liu et al., 2022)	97.70	92.10	92.10
18		PaDiM (Defard et al., 2021)	97.50	92.10	92.10
19		DR/EM (Zavrtanik et al., 2021)	97.31	–	–
20		DR/EM+SSPCAB (Ristea et al., 2021)	97.21	–	–
21		DR/EM+SSMCTB (Madan et al., 2022)	97.21	–	–
22		STPM (Wang et al., 2021a)	97.09	92.10	92.10
23		AnoSeg (Song et al., 2021)	96.87	–	–
24		GCPF (Wan et al., 2022a)	96.86	–	–
25		NSA (Schlitter et al., 2021)	96.28	–	–
26		SPADE (Cohen & Hoshen, 2020)	96.03	91.70	91.70
27		CutPaste (Li et al., 2021)	95.97	–	–
28		Patch SVDD (Yi & Yoon, 2020)	95.71	–	–
29		PFM (Wan et al., 2022b)	93.03	93.00	93.00
30		FCDD (Liznerski et al., 2021)	91.60	–	–
31		Glancing Patch (Wang et al., 2021b)	90.73	96.10	96.10
32		AE-SSIM (Bergmann et al., 2021a)	86.87	39.20	39.20
33		Visually Explaining VAE (Liu et al., 2019)	86.07	–	–
34		AE-L2 (Bergmann et al., 2021a)	81.73	30.60	30.60

The AUC is averaged over all classes of the MVTEC anomaly dataset



(a) Difficulty for detection.

(b) Difficulty for segmentation.

Fig. 24 Difficulty for different categories per method. To determine the difficulty, the binned AUC value is colored for each method. The order of the methods is determined by the ranking for the task. The method

on the leftmost side is the best method for the task. The order of the categories is determined by the average over all methods. The upper category is the overall most difficult category across all methods

Table 4 The methods that reported performance for both detection and segmentation are ranked in this table

Rank	Method	Rank on image level	Rank on pixel level	Average Rank
1	MemSeg (Yang et al., 2022)	1	2	1.50
2	PNI (Bae et al., 2022)	3	1	2.00
3	N-pad (Jang et al., 2022)	4	3	3.50
4	CFA (Lee et al., 2022)	2	5	3.50
5	FastFlow (Yu et al., 2021)	5	7	6.00
6	STPMDisc (Yamada et al., 2022)	10	5	7.50
7	PatchCore (Roth et al., 2021)	6	10	8.00
8	CFLOW-AD (Gudovskiy et al., 2021)	12	4	8.00
9	FAPM (Kim et al., 2022)	7	11	9.00
10	MSPB (Tsai et al., 2022)	13	9	11.00
11	Reverse Distillation (Deng & Li, 2022)	11	12	11.50
12	DRÆM+SSPCAB (Ristea et al., 2021)	8	16	12.00
13	DRAEM+SSMCTB (Madan et al., 2022)	9	16	12.50
14	FYD (Zheng et al., 2021)	17	8	12.50
15	SOMAD (Li et al., 2021)	15	13	14.00
16	DRÆM (Zavrtanik et al., 2021)	14	15	14.50
17	EdgRec (Liu et al., 2022)	16	14	15.00
18	NSA (Schlüter et al., 2021)	19	19	19.00
19	AnoSeg (Song et al., 2021)	21	18	19.50
20	CutPaste (Li et al., 2021)	20	20	20.00
21	PFM (Wan et al., 2022b)	18	22	20.00

The order is determined by the average rank of the respective tasks. MemSeg is the best method according to this benchmark, PNI is ranked second. N-pad follows in third place. Given the same rank, the order of presentation was based on the higher AUC average

els in terms of the number of parameters, which can lead to higher hardware requirements.

6 Discussion

More data and metrics needed

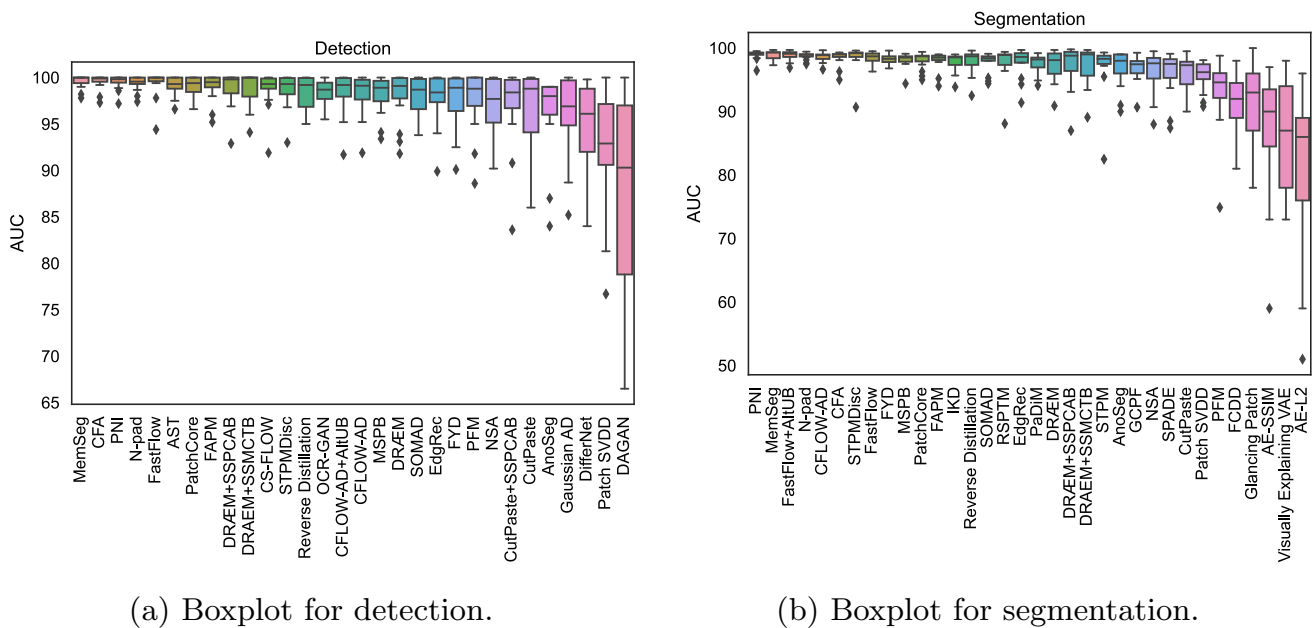


Fig. 25 Comparison of box plots for the methods. The plot shows the AUC values across all categories

The problem of industrial defect detection is a relatively recent one, and has arisen largely as a result of the MVTec AD. Compared to other areas of computer vision where many datasets are available for benchmarking, the benchmark on MVTec AD for industrial defect detection is the largest currently available. There were few alternatives to this dataset in the past. The advantage is that this dataset is relatively large, with over 5000 images from a variety of products, but there is always the suspicion that overfitting may occur on the test data if only this dataset is considered.

Nevertheless, more datasets have been made available that also contain defects in production pieces in high resolution. For the development of robust methods, which also achieve reliable results independent of a concrete dataset, it is necessary to include these datasets in the benchmark.

For this reason, in particular, and also because the current metrics seem saturated, it is necessary that the community considers other datasets and agrees on some benchmark datasets. We have listed some datasets such as MVTec 3D (Bergmann et al., 2021b) or OLP (Rippel et al., 2022) that could be used as alternatives, but we are also convinced that more such datasets will be proposed in the future. This is necessary to develop robust methods that deal well with highly unbalanced data sets with long-tailed distributions or highly noisy data.

New model classes and approaches in the field of anomaly detection: Normalizing Flows and unsupervised image segmentation

Looking at the different model classes, Normalizing Flows stands out as a new model category. This places an approach in the field of anomaly detection, which was popularized in

2014/2015 (Rezende & Mohamed, 2015; Dinh et al., 2014; Kobyzev et al., 2021) and has no direct relation to classical anomaly detection methods. Common approaches based on feature extraction with neural networks also remain successful, but have the difficulty that they cannot be learned end-to-end. Multiple steps are required to ensure defect detection in this case.

Many methods in this survey use artificial anomalies to learn image segmentation. Recently, unsupervised image segmentation *without* artificial labels has gained great importance in computer vision research. (Kim et al., 2020; Xia & Kulis, 2017; Croitoru et al., 2019). Caron et al. (2021) present a self-supervised method (DINO) that uses knowledge distillation and, among other things, small patches in a vision transformer to learn unsupervised image segmentation. TokenCut (Wang et al., 2022) interprets the image patches of a self-supervised ViT as a fully connected graph to learn an image segmentation. Finally, CutLER (Wang et al., 2023) sets a new benchmark in unsupervised image segmentation, performing better than supervised methods on some datasets.

These developments raise the question of whether learning based on synthetic anomalies is necessary. Although it works well, as the benchmark shows, it may be attractive and more general if images can be segmented without artificial anomalies. It is up to the community to apply the progress made in this area of research to the problem at hand.

Models from other domains such as medicine

The problem of subtle anomalies also exists in other fields, such as medicine. In this field, a variety of methods has been developed to detect anomalies in unsupervised settings.

When looking at the research done so far for defects on industrial images, it is noticeable that there is little exchange between the two related fields of research. It seems appropriate that not only models explicitly developed for MVTec AD are included in the benchmarks, but also models that provide reliable results for the related problem in medicine.

Zero-shot, few-shot and foundation models

Foundation models (Bommasani et al., 2021) (CLIP, BERT, DALL-E, GPT-3,...) represent a fundamental paradigm shift in the field of machine learning. These models are trained unsupervised on large amounts of data and offer remarkable possibilities to solve tasks for which they have not been explicitly trained.

In the field of image processing, this means that models trained to classify images based on descriptive text, for example, can perform ImageNet classification just as well as networks trained specifically for this task. As a prominent example, CLIP (Radford et al., 2021) achieves a top-1 accuracy of 76.2% on ImageNet without ever having seen an ImageNet image. This is equivalent to the accuracy of a ResNet-50 trained explicitly on 1.28 million images. While CLIP was one of the first methods to learn image classification from text, more recent models such as CoCa (Yu et al., 2022) have already achieved 86.3% top-1 accuracy in the zero-shot setting.

For semantically varying anomalies, as in the *k-classes-out* setting, these models have already been used successfully (Fort et al., 2021; Esmailpour et al., 2021) and the question is how and how successfully these models can be used for subtle anomalies.

The models work even more reliably when they are used in a few-shot setting, i.e., when they are trained on a few images of each class. This also applies to the area of anomaly detection: It is often the case that only a few labels are necessary to ensure reliable anomaly detection (Ruff et al., 2020b). Few labels are easier to obtain in practice than a fully annotated dataset. Additionally, there is evidence from other areas of anomaly detection that supervised learning methods can also generalize to previously unknown anomalies (Ruff et al., 2020a; Diers & Pigorsch, 2022; Hendrycks et al., 2018). Self-supervised learning methods also rely on known artificial anomalies to ensure that future anomalies can be detected. While it is a comparatively simple solution to use supervised learning methods with few labels, the importance of this approach for practical use should not be underestimated.

Improved representations based on representation learning

Overall, it is a challenge to extract relevant features for defect detection from the images. Currently, the methods mostly rely on widely used architectures such as ResNets, EfficientNets or Vision Transformer which reliably perform the feature extraction. Nevertheless, it is common in transfer learning to adapt the pre-trained models to the new dataset. This step is missing for most methods.

In general, it is also possible in the unsupervised setting to adapt feature extraction to the dataset at hand and create a workflow analogous to supervised transfer learning. The research field of representation learning (Bengio et al., 2013; Ericsson et al., 2022; Jaiswal et al., 2021b) develops methods to extract better embeddings for images. Self-supervised learning has the same goal. Here, the learned features are more customized to the dataset at hand and are less generic than is the case with pre-trained models based on ImageNet. Focus Your Distribution (Zheng et al., 2021) is a work in this survey that exploits representation learning methods.

Transferring advances in Explainable AI

A further transfer of research achievements can take place from other areas of Deep Learning. This includes advances to Explainable AI (XAI). The approach of XAI is to take black box models and visualize the classification decision (Adadi & Berrada, 2018). It is thus possible to take arbitrary models to localize the defects. The models do not have to be developed for image segmentation and can nevertheless locate defects on the image.

A common approach is LIME (Ribeiro et al., 2016). LIME removes superpixels from the image and observes the change in the classification result. Based on this data, a linear model is estimated, which estimates the classification result depending on the presence of a superpixel. The coefficients of the regression model represent the relevance of the superpixel to the neural network. A follow-up work by the authors of LIME proposes Anchors. Anchors have a similar principle, but allow a more precise localization of the relevant features for the prediction result (Ribeiro et al., 2018). The decision of the model can also be visualized with the help of the gradient. These include Grad-CAM (Selvaraju et al., 2017), Grad-CAM++ (Chattopadhyay et al., 2017) or HiResCam (Draelos & Carin, 2020).

7 Conclusion

This survey aims to give a comprehensive overview of different methods for industrial defect detection. For this purpose, the models were divided into groups and presented individually.

The results of the methods are remarkable. The field is still relatively young, which makes the achievements of the research community even more impressive. However, it is an open challenge to establish further benchmarks, as the datasets currently considered seem only partially suitable for further progress in the field. This is because the metrics for many methods are at a very high level, making it difficult to measure progress. In the future, it will be important to

include other major trends in computer vision and to include more datasets in the evaluation of new methods.

Funding Open Access funding enabled and organized by Projekt DEAL.

Data Availability The data of this survey is available on Github: <https://github.com/jandiers/mvtec-results>. It is taken from the original publications of the methods. The download of the mentioned datasets is possible via the links in the footnote.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by/4.0/>.

References

- Adadi, A., & Berrada, M. (2018). Peeking inside the black-box: A survey on explainable artificial intelligence (XAI). *IEEE Access*, 6, 52138–52160.
- Bae, J., Lee, J.-H., & Kim, S. (2022). Image anomaly detection and localization with position and neighborhood information. [arXiv:2211.12634](https://arxiv.org/abs/2211.12634).
- Bengio, Y., Courville, A., & Vincent, P. (2013). Representation learning: A review and new perspectives. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 35(8), 1798–1828.
- Bergman, L., & Hoshen, Y. (2020). Classification-based anomaly detection for general data. [arXiv:2005.02359](https://arxiv.org/abs/2005.02359).
- Bergman, L., Cohen, N., & Hoshen, Y. (2020). Deep nearest neighbor anomaly detection. [arXiv:2002.10445](https://arxiv.org/abs/2002.10445).
- Bergmann, P., Fauser, M., Sattlegger, D., & Steger, C. (2020). Uninformed students: Student-teacher anomaly detection with discriminative latent embeddings. In *2020 IEEE/CVF conference on computer vision and pattern recognition (CVPR)* (pp. 4182–4191). ISBN 2575-7075.
- Bergmann, P., Jin, X., Sattlegger, D., & Steger, C. (2021b). The MVTEC 3D-Ad dataset for unsupervised 3D anomaly detection and localization. [arXiv:2112.09045](https://arxiv.org/abs/2112.09045).
- Bergmann, P., Löwe, S., Fauser, M., Sattlegger, D., & Steger, C. (2018). Improving unsupervised defect segmentation by applying structural similarity to autoencoders. [arXiv:1807.02011](https://arxiv.org/abs/1807.02011).
- Bergmann, P., Batzner, K., Fauser, M., Sattlegger, D., & Steger, C. (2021). The MVTEC anomaly detection dataset: A comprehensive real-world dataset for unsupervised anomaly detection. *International Journal of Computer Vision*, 129(4), 1038–1059.
- Bl'azquez-Garc'ia, A., Conde, A., Mori, U., & Lozano, J. A. (2020). A review on outlier/anomaly detection in time series data. *ACM Computing Surveys (CSUR)*, 54, 1–33.
- Bommasani, R., Hudson, D. A., & Liang, P. (2021). On the opportunities and risks of foundation models. [arXiv:2108.07258](https://arxiv.org/abs/2108.07258).
- Breunig, M. M., Kriegel, H.-P., Ng, R. T., & Sander, J. (2000). LOF: Identifying density-based local outliers. In *SIGMOD '00*.
- Bucilua, C., Caruana, R., & Niculescu-Mizil, A. (2006). Model compression. In *Proceedings of the 12th ACM SIGKDD international conference on Knowledge discovery and data mining—KDD '06* (pp. 535–550). ACM Press.
- Cao, Y., Wan, Q., Shen, W., & Gao, L. (2022). Informative knowledge distillation for image anomaly segmentation. *Knowledge-Based Systems*, 248, 108846.
- Caron, M., Touvron, H., Misra, I., Jégou, H., Mairal, J., Bojanowski, P., & Joulin, A. (2021). Emerging properties in self-supervised vision transformers. [arXiv:2104.14294](https://arxiv.org/abs/2104.14294).
- Carrera, D., Manganini, F., Boracchi, G., & Lanzarone, E. (2017). Defect detection in SEM images of nanofibrous materials. *IEEE Transactions on Industrial Informatics*, 13(2), 551–561.
- Chalapathy, R., & Chawla, S. (2019). Deep learning for anomaly detection: A survey. [arXiv:1901.03407](https://arxiv.org/abs/1901.03407).
- Chattopadhyay, A., Sarkar, A., Howlader, P., & Balasubramanian, V. N. (2017). Grad-cam++: Generalized gradient-based visual explanations for deep convolutional networks. [arXiv:1710.11063](https://arxiv.org/abs/1710.11063).
- Chromiński, K., & Tkacz, M. (2010). Comparison of outlier detection methods in biomedical data. *Journal of Medical Informatics and Technologies*, 16, 89–94.
- Cohen, N., & Hoshen, Y. (2020). Sub-image anomaly detection with deep pyramid correspondences. [arXiv:2005.02357v3](https://arxiv.org/abs/2005.02357v3).
- Croitoru, I., Bogolin, S.-V., & Leordeanu, M. (2019). Unsupervised learning of foreground object detection. [arXiv:1808.04593](https://arxiv.org/abs/1808.04593).
- de Albuquerque Filho, J. E., Brandão, L. C. P., Fernandes, B. J. T., & Maciel, A. M. A. (2022). A review of neural networks for anomaly detection. *IEEE Access*, 10, 112342–112367.
- Defard, T., Setkov, A., Loesch, A., & Audigier, R. (2021). Padim: A patch distribution modeling framework for anomaly detection and localization. In A. Del Bimbo, R. Cucchiara, S. Sclaroff, G. M. Farinella, T. Mei, M. Bertini, H. J. Escalante, R. Vezani (Eds.), *Pattern recognition. ICPR international workshops and challenges, volume 12664 of Springer eBook Collection* (pp. 475–489). Springer.
- Deng, H., & Li, X. (2022). Anomaly detection via reverse distillation from one-class embedding. *CVPR*. [arXiv:2201.10703](https://arxiv.org/abs/2201.10703).
- Deng, J., Dong, W., Socher, R., Li, L.-J., Li, K., & Li, F.-F. (2009). Imagenet: A large-scale hierarchical image database. In *2009 IEEE conference on computer vision and pattern recognition* (pp. 248–255). ISBN 1063-6919.
- Di Mattia, F., Galeone, P., de Simoni, M., & Ghelfi, E. (2019). A survey on GANS for anomaly detection. [arXiv:1906.11632](https://arxiv.org/abs/1906.11632).
- Diers, J., & Pigorsch, C. (2022). *Out-of-distribution detection using outlier detection methods* (pp. 15–26). Springer.
- Dinh, L., Krueger, D., & Bengio, Y. (2014). Non-linear independent components estimation: Nice. [arXiv:1410.8516](https://arxiv.org/abs/1410.8516).
- Doersch, C., Gupta, A., & Efros, A. A. (2015). Unsupervised visual representation learning by context prediction. [arXiv:1505.05192](https://arxiv.org/abs/1505.05192).
- Draeos, R. L., & Carin, L. (2020). Use hirescam instead of grad-cam for faithful explanations of convolutional neural networks. [arXiv:2011.08891](https://arxiv.org/abs/2011.08891).
- Ericsson, L., Gouk, H., Loy, C. C., & Hospedales, T. M. (2022). Self-supervised representation learning: Introduction, advances, and challenges. *IEEE Signal Processing Magazine*, 39(3), 42–62.
- Esmailpour, S., Liu, B., Robertson, E., & Shu, L. (2021). Zero-shot out-of-distribution detection based on the pre-trained model clip. [arXiv:2109.02748](https://arxiv.org/abs/2109.02748).
- Fort, S., Ren, J., & Lakshminarayanan, B. (2021). Exploring the limits of out-of-distribution detection. [arXiv:2106.03004](https://arxiv.org/abs/2106.03004).
- Gaspar, J., Catumbela, E., Marques, B., & Freitas, A. (2011). A systematic review of outliers detection techniques in medical data—preliminary study (pp. 575–582). https://www.researchgate.net/profile/juliano-gaspar/publication/221334605_a_systematic_review_of_outliers_detection_techniques_in_medical_data_-_preliminary_study

- Gidaris, S., Singh, P., & Komodakis, N. (2018). Unsupervised representation learning by predicting image rotations. *ICLR 2018*.
- Golan, I., & El-Yaniv, R. (2018). Deep anomaly detection using geometric transformations. *Advances in Neural Information Processing Systems*, 31.
- Gourisaria, M. K., Agrawal, R., Harshvardhan, G. M., Pandey, M., & Rautaray, S. S. (2021). Application of machine learning in industry 4.0. In *Machine learning: Theoretical foundations and practical applications* (pp. 57–87). Springer.
- Gou, J., Yu, B., Maybank, S. J., & Tao, Dacheng. (2021). Knowledge distillation: A survey. *International Journal of Computer Vision*, 129(6), 1789–1819.
- Gudovskiy, D., Ishizaka, S., & Kozuka, K. (2021). Cflow-ad: Real-time unsupervised anomaly detection with localization via conditional normalizing flows. [arXiv:2107.12571](https://arxiv.org/abs/2107.12571).
- Gupta, M., Gao, J., Aggarwal, C., & Han, J. (2014). Outlier detection for temporal data: A survey. *IEEE Transactions on Knowledge and Data Engineering*. <https://www.semanticscholar.org/paper/Outlier-Detection-for-Temporal-Data%3A-A-Survey-Gupta-Gao/43d75d3a22db904d052d4c435e2d1f22be3887e0>
- Hautamaki, V., Karkkainen, I., & Franti, P. (2004). Outlier detection using k-nearest neighbour graph. In *Proceedings of the 17th international conference on pattern recognition, 2004. ICPR 2004*. IEEE.
- He, K., Zhang, X., Ren, S., & Sun, J. (2015). Deep residual learning for image recognition. [arXiv:1512.03385v1](https://arxiv.org/abs/1512.03385v1).
- Hendrycks, D., & Dietterich, T. (2019). Benchmarking neural network robustness to common corruptions and perturbations. [arXiv:1903.12261](https://arxiv.org/abs/1903.12261)
- Hendrycks, D., Mazeika, M., & Dietterich, T. (2018). Deep anomaly detection with outlier exposure. [arXiv:1812.04606](https://arxiv.org/abs/1812.04606).
- Hinton, G., Vinyals, O., & Dean, J. (2015). Distilling the knowledge in a neural network. *NIPS 2014*.
- Hodge, V. J., & Austin, J. (2004). A survey of outlier detection methodologies. *Artificial Intelligence Review*.
- Hozdić, E. (2015). Smart factory for industry 4.0: A review. *Journal of Modern Manufacturing Systems and Technology*, 7(1), 28–35.
- Huang, Y., Qiu, C., & Yuan, K. (2020). Surface defect saliency of magnetic tile. *The Visual Computer*, 36(1), 85–96.
- Jaiswal, A., Babu, A. R., Zadeh, M. Z., Banerjee, D., & Makedon, F. (2021). A survey on contrastive self-supervised learning. *Technologies*, 9(1), 2.
- Jaiswal, A., Babu, A. R., Zadeh, M. Z., Banerjee, D., & Makedon, F. (2021). A survey on contrastive self-supervised learning. *Technologies*, 9(1), 2.
- Jang, J., Hwang, E., & Park, S.-H. (2022). N-pad: Neighboring pixel-based industrial anomaly detection. [arXiv:2210.08768](https://arxiv.org/abs/2210.08768).
- Kang, C. W., Ramzan, M. B., Sarkar, B., & Imran, M. (2018). Effect of inspection performance in smart manufacturing system based on human quality control system. *The International Journal of Advanced Manufacturing Technology*, 94(9–12), 4351–4364.
- Kendall, A., & Gal, Y. (2017). What uncertainties do we need in Bayesian deep learning for computer vision? [arXiv:1703.04977](https://arxiv.org/abs/1703.04977).
- Kim, Y., Jang, H., Lee, D., & Choi, H.-J. (2022). Altub: Alternating training method to update base distribution of normalizing flow for anomaly detection. [arXiv:2210.14913](https://arxiv.org/abs/2210.14913).
- Kim, W., Kanezaki, A., & Tanaka, M. (2020). Unsupervised learning of image segmentation based on differentiable feature clustering. [arXiv:2007.09990](https://arxiv.org/abs/2007.09990).
- Kim, J.-H., Kim, D.-H., Yi, S., & Lee, T. (2021). Semi-orthogonal embedding for efficient unsupervised anomaly segmentation. [arXiv:2105.14737](https://arxiv.org/abs/2105.14737).
- Kim, D., Park, C., Cho, S., & Lee, S. (2022). FAPM: Fast adaptive patch memory for real-time industrial anomaly detection. [arXiv:2211.07381](https://arxiv.org/abs/2211.07381).
- Kobyzev, I., Prince, S. J. D., & Brubaker, M. A. (2021). Normalizing flows: An introduction and review of current methods. [arXiv:1908.09257](https://arxiv.org/abs/1908.09257).
- Krasin, I., Duerig, T., Alldrin, N., Ferrari, V., Abu-El-Hajja, S., Kuznetsova, A., Rom, H., Uijlings, J., Popov, S., Kamali, S., Mallocci, M., Pont-Tuset, J., Veit, A., Belongie, S., Gomes, V., Gupta, A., Sun, C., Chechik, G., Cai, D., Feng, Z., Narayanan, D., & Murphy, K. (2017). Openimages: A public dataset for large-scale multi-label and multi-class image classification.
- Lee, S., Lee, S., & Song, B. C. (2022). CFA: Coupled-hypersphere-based feature adaptation for target-oriented anomaly localization. [arXiv:2206.04325](https://arxiv.org/abs/2206.04325).
- Li, N., Jiang, K., Ma, Z., Wei, X., Hong, X., & Gong, Y. (2021). Anomaly detection via self-organizing map. [arXiv:2107.09903](https://arxiv.org/abs/2107.09903).
- Li, C.-L., Sohn, K., Yoon, J., & Pfister, T. (2021). Cutpaste: Self-supervised learning for anomaly detection and localization. [arXiv:2104.04015](https://arxiv.org/abs/2104.04015).
- Liang, Y., Zhang, J., Zhao, S., Wu, R., Liu, Y., & Pan, S. (2022). Omni-frequency channel-selection representations for unsupervised anomaly detection. [arXiv:2203.00259](https://arxiv.org/abs/2203.00259).
- Lilhore, U. K., Simaiya, S., Kaur, A., Prasad, D., Khurana, M., Verma, D. K., & Hassan, A. (2022). Impact of deep learning and machine learning in industry 4.0: Impact of deep learning. In V. Bali, V. Bhatnagar, D. Aggarwal, S. Bali (Eds.), *Cyber-physical, IoT, and autonomous systems in industry 4.0* (pp. 179–197). CRC Press. ISBN 9781003146711.
- Liu, R., Lehman, J., Molino, P., Such, F. P., Frank, E., Sergeev, A., & Yosinski, J. (2018). An intriguing failing of convolutional neural networks and the coordconv solution. [arXiv:1807.03247](https://arxiv.org/abs/1807.03247).
- Liu, T., Li, B., Zhao, Z., Xiao, D., Jiang, B., & Geng, L. (2022). Reconstruction from edge image combined with color and gradient difference for industrial surface anomaly detection. [arXiv:2210.14485](https://arxiv.org/abs/2210.14485).
- Liu, W., Li, R., Zheng, M., Karanam, S., Wu, Z., Bhanu, B., Radke, R. J., & Camps, O. (2019). Towards visually explaining variational autoencoders. [arXiv:1911.07389](https://arxiv.org/abs/1911.07389).
- Liznerski, P., Ruff, L., Vandermeulen, R. A., Franks, B. J., Kloft, M., & Müller, K.-R. (2021). Explainable deep one-class classification. In *ICLR 2021*.
- Madan, N., Ristea, N.-C., Ionescu, R. T., Nasrollahi, K., Khan, F. S., Moeslund, T. B., & Shah, M. (2022). Self-supervised masked convolutional transformer block for anomaly detection. [arXiv:2209.12148](https://arxiv.org/abs/2209.12148).
- Minaee, S., Boykov, Y. Y., Porikli, F., Plaza, A. J., Kehtarnavaz, N., & Terzopoulos, D. (2021). Image segmentation using deep learning: A survey. *IEEE Transactions on Pattern Analysis and Machine Intelligence*.
- Mishra, P., Verk, R., Fornasier, D., Piciarelli, C., & Foresti, G. L. (2021). VT-ADL: A vision transformer network for image anomaly detection and localization. *KD-*, 01–06. [arXiv:2104.10036](https://arxiv.org/abs/2104.10036).
- Mu, N., & Gilmer, J. (2019). Mnist-c: A robustness benchmark for computer vision. [arXiv:1906.02337](https://arxiv.org/abs/1906.02337).
- Napoletano, P., Piccoli, F., & Schettini, R. (2018). Anomaly detection in nanofibrous materials by cnn-based self-similarity. *Sensors (Basel, Switzerland)*, 18(1).
- Noroozi, M., & Favaro, P. (2016). Unsupervised learning of visual representations by solving jigsaw puzzles. In B. Leibe, J. Matas, N. Sebe, M. Welling (Eds.), *Computer vision - ECCV 2016, Lecture Notes in Computer Science* (pp. 69–84). Springer. ISBN 978-3-319-46466-4.
- Pang, G., Shen, C., Cao, L., & den van Hengel, A. (2022). Deep learning for anomaly detection. *ACM Computing Surveys*, 54(2), 1–38. [arXiv:2007.02500](https://arxiv.org/abs/2007.02500).
- Radford, A., Kim, J. W., Hallacy, C., Ramesh, A., Goh, G., Agarwal, S., Sastry, G., Askell, A., Mishkin, P., Clark, J., Krueger, G., & Sutskever, I. (2021). Learning transferable visual models from natural language supervision. [arXiv:2103.00020](https://arxiv.org/abs/2103.00020).

- Rezende, D. J., & Mohamed, S. (2015). Variational inference with normalizing flows. [arXiv:1505.05770](#).
- Ribeiro, M. T., Singh, S., & Guestrin, C. (2016). Why should I trust you? Explaining the predictions of any classifier. [arXiv:1602.04938](#).
- Ribeiro, M. T., Singh, S., & Guestrin, C. (2018). Anchors: High-precision model-agnostic explanations. In *AAAI conference on artificial intelligence (AAAI)* (pp. 1–9).
- Rippel, O., Mertens, P., & Merhof, D. (2020). Modeling the distribution of normal data in pre-trained deep features for anomaly detection. [arXiv:2005.14140](#).
- Rippel, O., Zwinge, C., & Merhof, D. (2022). Increasing the generalization of supervised fabric anomaly detection methods to unseen fabrics. *Sensors*, 22(13), 4750.
- Ristea, N.-C., Madan, N., Ionescu, R. T., Nasrollahi, K., Khan, F. S., Moeslund, T. B., & Shah, M. (2021). Self-supervised predictive convolutional attentive block for anomaly detection. [arXiv:2111.09099](#).
- Roth, K., Pemula, L., Zepeda, J., Schölkopf, B., Brox, T., & Gehler, P. (2021). Towards total recall in industrial anomaly detection. [arXiv:2106.08265v1](#).
- Rudolph, M., Wandt, B., & Rosenhahn, B. (2020). Same same but different: Semi-supervised defect detection with normalizing flows. [arXiv:2008.12577](#).
- Rudolph, M., Wehrbein, T., Rosenhahn, B., & Wandt, B. (2021). Fully convolutional cross-scale-flows for image-based defect detection. [arXiv:2110.02855](#).
- Rudolph, M., Wehrbein, T., Rosenhahn, B., & Wandt, B. (2022). Asymmetric student-teacher networks for industrial anomaly detection. [arXiv:2210.07829](#).
- Ruff, L., Kauffmann, J. R., Vandermeulen, R. A., Montavon, G., Samek, W., Kloft, M., Dietterich, T. G., & Müller, K.-R. (2021). A unifying review of deep and shallow anomaly detection. *Proceedings of the IEEE*, 109(5), 756–795. [arXiv:2009.11732](#).
- Ruff, L., Vandermeulen, R. A., Franks, B. J., Müller, K.-R., & Kloft, M. (2020). Rethinking assumptions in deep anomaly detection. [arXiv:2006.00339](#).
- Ruff, L., Vandermeulen, R. A., Franks, B. J., Müller, K.-R., & Kloft, M. (2020a). Rethinking assumptions in deep anomaly detection. [arXiv:2006.00339](#).
- Ruff, L., Vandermeulen, R., Goernitz, N., Deecke, L., Ahmed, S., Siddiqui, A., Binder, A., Müller, E., & Kloft, M. (2018). Deep one-class classification. In *International conference on machine learning* (pp. 4393–4402). <https://proceedings.mlr.press/v80/ruff18a.html>
- Ruff, L., Vandermeulen, R., Goernitz, N., Deecke, L., Siddiqui, S. A., Binder, A., Müller, E., & Kloft, M. (2018). Deep one-class classification. In *International conference on machine learning* (pp. 4393–4402). <https://proceedings.mlr.press/v80/ruff18a.html>
- Ruff, L., Vandermeulen, R. A., Goernitz, N., Binder, A., Müller, E., Müller, K.-R., & Kloft, M. (2020b). *Deep semi-supervised anomaly detection*.
- Schlüter, H. M., Tan, J., Hou, B., & Kainz, B. (2021). Self-supervised out-of-distribution detection and localization with natural synthetic anomalies (NSA). [arXiv:2109.15222](#).
- Schölkopf, B., Williamson, R. C., Smola, A., Shawe-Taylor, J., & Platt, J. (1999). *Support vector method for novelty detection*.
- Selvaraju, R. R., Cogswell, M., Das, A., Vedantam, R., Parikh, D., & Batra, D. (2017). Grad-cam: Visual explanations from deep networks via gradient-based localization. In *2017 IEEE international conference on computer vision (ICCV)* (pp. 618–626). IEEE.
- Shahid, N., Naqvi, I., & Qaisar, Saad B. (2012). Characteristics and classification of outlier detection techniques for wireless sensor networks in Harsh environments: A survey. *Artificial Intelligence Review*.
- Singh, K., & Shuchita, U. (2012). Outlier detection: Applications and techniques. *International Journal of Computer Science Issues (IJCSI)*, 9(1), 307.
- Song, J., Kong, K., Park, Y.-I., Kim, S.-G., & Kang, S.-J. (2021). *AnoSeg: Anomaly segmentation network using self-supervised learning*. https://www.researchgate.net/publication/355141739_AnoSeg_Anomaly_Segmentation_Network_Using_Self-Supervised_Learning
- Song, K., & Yan, Y. (2013). A noise robust method based on completed local binary patterns for hot-rolled steel strip surface defects. *Applied Surface Science*, 285, 858–864.
- Tabernik, D., Šela, S., Skvarč, J., & Skočaj, D. (2020). Segmentation-based deep-learning approach for surface-defect detection. [arXiv:1903.08536](#).
- Taha, A., & Hadi, A. S. (2020). Anomaly detection methods for categorical data. *ACM Computing Surveys*, 52(2), 1–35.
- Tan, M., & Le, Q. (2019). Efficientnet: Rethinking model scaling for convolutional neural networks. In *International conference on machine learning*. [arXiv:1905.11946v5](#).
- Tang, T.-W., Kuo, W.-H., Lan, J.-H., Ding, C.-F., Hsu, H., & Young, H.-T. (2020). Anomaly detection neural network with dual auto-encoders GAN and its industrial inspection applications. *Sensors*, 20(12), 3336.
- Tax, D. M. J., & Duin, R. P. W. (2004). Support vector data description. *Machine Learning*, 54(1), 45–66.
- Thakkar, P., Vala, J. B., & Prajapati, V. (2016). Survey on outlier detection in data stream. *International Journal of Computer Applications*.
- Touvron, H., Cord, M., Sablayrolles, A., Synnaeve, G., & Jégou, H. (2021). Going deeper with image transformers. [arXiv:2103.17239](#).
- Tsai, C. C., Wu, T. H., & Lai, S. H. (2022). Multi-scale patch-based representation learning for image anomaly detection and segmentation. In *2022 IEEE/CVF winter conference on applications of computer vision (WACV)* (pp. 3065–3073). ISBN 2642-9381.
- Wan, Q., YunKang, C., Gao, L., Weiming, S., & Li, X. (2022c). Position encoding enhanced feature mapping for image anomaly detection.
- Wang, X., Girdhar, R., Yu, S. X., & Misra, I. (2023). Cut and learn for unsupervised object detection and instance segmentation. [arXiv:2301.11320](#).
- Wang, G., Han, S., Ding, E., & Huang, D. (2021a). Student-teacher feature pyramid matching for anomaly detection. [arXiv:2103.04257](#).
- Wang, Y., Shen, X., Yuan, Y., Yuming, D., Li, M., Hu, S. X., Crowley, J. L., & Vaufraydaz, D. (2022). TokenCut: Segmenting objects in images and videos with self-supervised transformer and normalized cut. [arXiv:2209.00383](#).
- Wang, S., Wu, L., Cui, L., & Shen, Y. (2021b). Glancing at the patch: Anomaly localization with global and local feature comparison. In *2021 IEEE/CVF conference on computer vision and pattern recognition (CVPR)* (pp. 254–263). IEEE. ISBN 978-1-6654-4509-2.
- Wan, Q., Gao, L., Li, X., & Wen, L. (2022). Industrial image anomaly localization based on gaussian clustering of pretrained feature. *IEEE Transactions on Industrial Electronics*, 69(6), 6182–6192.
- Wan, Q., Gao, L., Li, X., & Wen, L. (2022). Unsupervised image anomaly detection and segmentation based on pre-trained feature mapping. *IEEE Transactions on Industrial Informatics*, 25, 1–10.
- Wang, Z., Bovik, A. C., Sheikh, H. R., & Simoncelli, E. P. (2004). Image quality assessment: From error visibility to structural similarity. *IEEE Transactions on Image Processing*, 13(4), 600–612.
- Wieler, M., & Hahn, T. (2007). Weakly supervised learning for industrial optical inspection.
- Xia, X., & Kulis, B. (2017). W-Net: A deep model for fully unsupervised image segmentation. [arXiv:1711.08506](#)

- Xu, H., Caramanis, C., & Sanghavi, S. (2010). Robust PCA via outlier pursuit. *Advances in Neural Information Processing Systems*, 23.
- Xu, F., Uszkoreit, H., Du, Y., Fan, W., Zhao, D., & Zhu, J. (2019). Explainable AI: A brief survey on history, research areas, approaches and challenges. *Lecture Notes in Computer Science*, 25, 563–574.
- Yamada, S., & Hotta, K. (2021). Reconstruction student with attention for student-teacher pyramid matching. [arXiv:2111.15376](#).
- Yamada, S., Kamiya, S., & Hotta, K. (2022). Reconstructed student-teacher and discriminative networks for anomaly detection. [arXiv:2210.07548](#).
- Yang, M., Wu, P., Liu, J., & Feng, H. (2022). MemSeg: A semi-supervised method for image surface defect detection using differences and commonalities. [arXiv:2205.00908](#).
- Yi, J., & Yoon, S. (2020). Patch SVDD: Patch-level svdd for anomaly detection and segmentation. [arXiv:2006.16067](#).
- Yu, J., Wang, Z., Vasudevan, V., Yeung, L., Seyedhosseini, M., & Wu, Y. (2022). Coca: Contrastive captioners are image-text foundation models. [arXiv:2205.01917](#).
- Yu, J., Zheng, Y., Wang, X., Li, W., Wu, Y., Zhao, R., & Wu, L. (2021). Fastflow: Unsupervised anomaly detection and localization via 2D normalizing flows. [arXiv:2111.07677](#).
- Zavrtanik, V., Kristan, M., & Skočaj, D. (2021). Draem: A discriminatively trained reconstruction embedding for surface anomaly detection. [arXiv:2108.07610](#).
- Zheng, Y., Wang, X., Deng, R., Bao, T., Zhao, R., & Wu, L. (2021). Focus your distribution: Coarse-to-fine non-contrastive learning for anomaly detection and localization. [arXiv:2110.04538](#).
- Zheng, Y., Wang, X., Qi, Y., Li, W., & Wu, L. (2022). Benchmarking unsupervised anomaly detection and localization.
- Zuehlke, D. (2010). Smartfactory—towards a factory-of-things. *Annual Reviews in Control*, 34(1), 129–138.

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.